Evaluating the Impact of Augmenting the VAEP Model with 360-Degree Freeze-Frame Spatial Context

Candidate Number: 291723

Abstract

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Curabitur convallis leo sit amet euismod finibus. Nullam vestibulum metus nec mauris varius, eu aliquam purus commodo. Sed faucibus ex nisl, id bibendum ligula ultrices nec. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nulla volutpat gravida sem, et consequat diam pretium ac. Donec condimentum pulvinar scelerisque. Orci varius natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Morbi elementum felis sapien, id euismod metus tincidunt vel. Mauris elementum eu neque elementum molestie. Morbi accumsan, nunc in malesuada semper, neque urna suscipit neque, non ornare velit purus id massa. Sed ut mi vel leo egestas euismod vel sit amet orci. Sed sed placerat odio. Mauris tincidunt id ex id commodo. Etiam hendrerit lacus et velit gravida, in volutpat enim aliquet. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia curae; Cras vel tortor sagittis, dignissim purus sit amet.

Table of Contents

1. Introduction 4
2. Research Objectives 5
3. Background 5
3.1. Valuing Actions by Estimating Probabilities (VAEP) 5
3.2. Related Work: Applications of Spatial Context to Action Value Metrics 7
3.2.1. Expected Goals (xG) 7
3.2.2. Expected Threat (xT) 7
3.2.3. Expected Possession Value (EPV) 8
3.2.4. Summary 9
3.3. Representing Spatial Context 9
3.3.1. Manual Spatial Feature Engineering 9
3.3.2. Spatial Embeddings: Set Transformer 9
4. Methodology 11
4.1. Datasets Outline 11
4.1.1. Limitations of Datasets 12
4.2. Data Collection 13
4.3. Data Preprocessing 13
4.4. Data Engineering 13
4.4.1. Manually-Engineered Spatial Context Features 13
4.4.2. Spatial Context Embeddings 14
4.5. Set Transformer Hyperparameter Tuning 17
4.5.1. Tree-Structured Parzen Estimatior (TPE) 18
4.5.2. Hyperband Pruning 19
4.6. Classifier Training 20
4.6.1. Logistic Regression 20
4.6.2. Random Forest 20
4.6.3. Gradient Boosting 21
4.6.4. Loss Function: Cross-Entropy Loss 21
4.6.5. Split Criterion: Gini impurity 22
4.7. Model Evaluation Metrics 22
4.7.1. Area Under the Receiver Operating Characteristic Curve (AUC-ROC) 22
4.7.2. Brier Score 22
4.7.3. Cross-Entropy Loss 23
4.7.4. Expected Calibration Error (ECE) 23
4.8. Model Interpretability 23
4.8.1. Permutation Importance 23
4.8.2. Partial Dependence Plots (PDPs) 23
4.8.3. k-Means Clustering 24
4.8.4. Uniform Manifold Approximation and Projection (UMAP) 25
5. Results 26
5.1. Set Transformer Training 26
5.2. Model Training 28
5.2.1. Feature Set Significance Tests 29
5.2.2. VAEP EDA 32
5.3. Model Interpretation 32
5.3.1. Permutation Importance 32
5.3.2. Partial Dependence Plots 35
5.3.3. K-means Clustering: Spatial Embeddings 36
5.3.4. UMAP 37

6. Discussion 37
7. Conclusion 37
References 38

1. Introduction

The adoption of analytics in association football has grown significantly in the last couple of decades (Goes et al., 2021; Olthof & Davis, 2025; Schaerlaeckens, 2025; Wakelam et al., 2022), enabling football clubs to leverage insights to inform tactical decisions, coaching and recruitment. Much of this innovation has involved the development of metrics used to determine the value provided by a particular action made by a player in terms of goal-scoring probabilities. One such metric is Valuing Actions by Estimating Probabilities (VAEP) (Decroos et al., 2019).

VAEP considers the probability of scoring or conceding within the next k events to assign a value between -2 and 2 to each action. VAEP, like other such metrics, typically uses action-centric features such as the action's start and end location and type (carry, pass, shot) in estimating scoring/conceding probabilties, but this approach does not explicitly model the wider spatial context (Sá-Freire, 2025). The spatial context - the position of teammates and opponents around the actor - of an action determines the options available to a player, showing viable passing options or space to carry into, as well as the proximity of opposition defenders applying pressure or blocking options.

The StatsBomb Open Data repository (StatsBomb, 2026a) provides events (e.g. carries, passes, shots) data for matches, along with *three-sixty* freeze-frames which indicate the positions of players within proximity of the acting player. Such data can be used to incorporate spatial context as a predictive feature. To illustrate this, **Figure 1** compares a pass event from a match with and without the surrounding spatial context. Without the spatial context, this pass appears to offer limited progression towards the opponent's goal. Traditional VAEP would consider how passes of this trajectory tend to result in scoring/conceding, but the addition of spatial context differentiates this particular pass from others of similar trajectory. **Figure 1** shows how the left hand side of the actor is blocked by multiple opponents (red) and this pass moves the ball into space on the right flank to open an attacking opportunity (in this case, a long-ranged shot)[1]. Spatial context could allow finer-grained differentiation, rather than aggregating all passes of this trajectory.

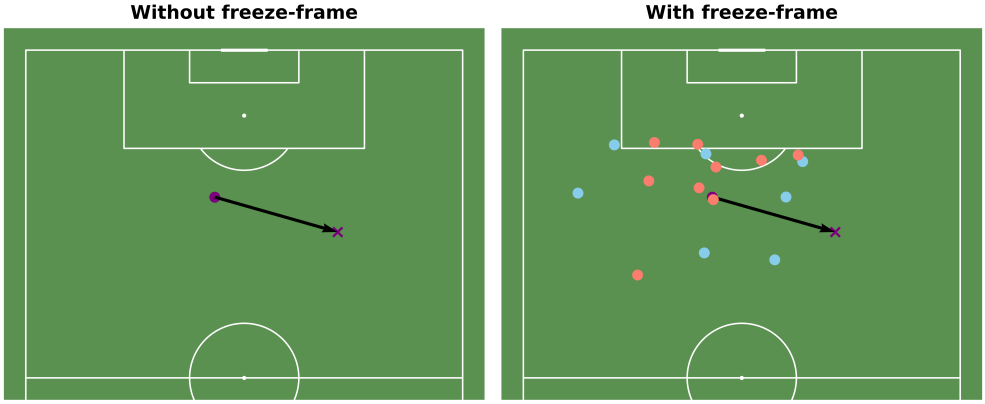


Figure 1: Pass Event With and Without Freeze-Frame

[1]The pass event id is '23b6b26a-293c-4d5e-9889-5a5ce0b7ad8c' and the subsequent long-ranged shot event id is 'd9d27d20-35bc-42bc-ac53-059ad519f781'

2. Research Objectives

The general aim of this project is to determine to what extent the addition of spatial context features enhances and alters the standard VAEP model. This entails the following objectives:

- Develop representations of spatial context from StatsBomb 360 freeze-frame data using feature engineering and deep learning techniques.
- Train and evaluate VAEP models, both with and without the inclusion of spatial context representations, using supervised classification algorithms.
- Compare and interpret these VAEP models using appropriate peformance metrics and an exploration of feature importance.

3. Background

In soccer analytics, *actions* refer to a *type of event* within a match which is caused by a player (e.g. a pass), with *events* being the broader set of significant occurrences in a match (e.g. passes, but also substitutions, fouls etc.) (Decroos, Bransen, Van Haaren, & Davis, 2019). Typical performance metrics focus on action outcomes, such as goals, assists and pass completion rates. Whilst such metrics can be used to determine the value of some actions and are useful for summarising player ability, soccer analytics has developed metrics to help generalise this notion of action value. Moreover, models used to assign value to actions have been enhanced by incorporating the locations of teammates and opponents, leading to improved performance and interpretability.

As such, this section will introduce the VAEP metric and outline how spatial context has been successfully incorporated into other action valuation metrics, motivating this project's research objectives.

3.1. Valuing Actions by Estimating Probabilities (VAEP)

Decroos, Bransen, Van Haaren, & Davis (2019) introduce Valuing Actions by Estimating Probabilities (VAEP). VAEP values actions in terms of the probability of scoring and the probability of conceding within the next k actions. Equation 1 defines the VAEP of an action a_i for team x.

$$V(a_i, x) = \Delta P_{scores}(a_i, x) - \Delta P_{concedes}(a_i, x), \quad V(a_i, x) \in [-2, 2] \tag{1}$$

If we consider a match as a sequence of m on-ball actions $[a_1, ..., a_m]$, we can view some game state S_i which corresponds to some subset of actions $[a_1, ..., a_i]$ and determine how the last action in the game state a_i changed the probability of team x scoring and conceding within the next k actions. The change in the probability of scoring (and conceding) from the previous game state $S_{i-1} = [a_1, ..., a_{i-1}]$ to the current game state S_i can be determined by simply finding their difference:

$$\begin{aligned} \Delta P_{\text{scores}}(a_i, x) &= P_{\text{scores}}(S_i, x) - P_{\text{scores}}(S_{i-1}, x) \\ \Delta P_{\text{concede}}(a_i, x) &= P_{\text{concede}}(S_i, x) - P_{\text{concede}}(S_{i-1}, x) \end{aligned} \tag{2}$$

These two probabilities are estimated using two supervised classification algorithms, one which estimates scoring, and the other conceding, within the next k actions. The feature sets used across these two models are identical, all that varies is the labels (scoring or conceding). Decroos, Bransen, Van Haaren, & Davis (2019) used a feature set which corresponds to the

Soccer Player Action Description Language (SPADL) format involving 9 features, most notably the action's start and end locations, and the type of action. The authors also add "complex features", such as the time elapsed between consecutive actions, as well as "game context features", such as the scoreline at the time of the action and the amount of time remaining.

Decroos, Bransen, Van Haaren, & Davis (2019) initially compared logistic regression, a random forest, XGBoost and CatBoost, finding CatBoost to perform best across both the scoring and conceding models in terms of the Brier Score and the Area Under the Receiver Operating Characteristic Curve (AUC-ROC). Decroos, Bransen, Van Haaren, & Davis (2019) also extend the VAEP metric by using it to provide player ratings, where the VAEP of each action in a player's set of actions A_p^T across some time frame T are considered cumulatively, and normalised per-90 minutes played during T.

$$\text{rating } (p) = \frac{90}{m} \sum_{a_i \in A_p^T} V(a_i) \tag{3}$$

The authors compare these VAEP player ratings against traditional metrics like goals and assists per-90, as well as market valuations of players. The authors found the top-10 highest players ranked by VAEP captured players typically considered to be amongst the best in the Premier League, despite them not appearing in the top-10 goals + assists per-90 ranking. The top-10 VAEP ranked list also had a higher overall market valuation than the top-10 goals + assists per-90 ranked list (€$1,110$ Million vs $947 Million$), which suggests VAEP more comprehensively represents player value compared to traditional metrics.

Decroos & Davis (2020) extend their previous work by improving model interpretability. Given that the initial findings from Decroos, Bransen, Van Haaren, & Davis (2019) suggest the use of gradient boosting algorithms for VAEP model classifiers, these models are more difficult to interpret than methods such as logistic regression. As such, Decroos & Davis (2020) built a generalised additive model (GAM) on the top-10 most important features from the original VAEP feature set, as determined by the average *gain* (for the present classifiers, this is the reduction of Gini impurity - see Section 4.6.5) that feature imports on the model across all splits involving that feature. The authors found a very small drop in classifier performance: a 0.2% increase in the normalised Brier score compared to XGBoost on the original VAEP feature set, and a 0.1 increase on the top-10 most important feature set. The authors argue the large gains in utility came from the added interpretability from being able to assess each univariate function which comprised their GAM model. This enables a direct observation of how each function contributes to its prediction across the domain of that function. For example, *distance to opponent goal* constituted one of the top-10 features, and figure 2 in Decroos & Davis (2020) shows how the goal scoring probability changes across the domain of this feature's function. Matching intuition, a higher goal scoring probability is observed for lower distances to the opponent's goal, with this probability dropping exponentially as distance increases. This added interpretability helps to demonstrate the greater tactical understanding which soccer analysts may benefit from when using a GAM-based VAEP model, and that such interpretability may be gained without a significant loss in performance.

One potential limitation of the VAEP model is that it lacks a representation of wider spatial context from the positions of teammates and opponents during actions (Sá-Freire, 2025), due to its use of event data only. Every action taken by a player exists within a spatial context which determines the options available to a player, influencing their decision making. The use of spatial context in VAEP models is also understudied, despite its addition having been successfully applied to a number of alternative action value metrics.

3.2. Related Work: Applications of Spatial Context to Action Value Metrics

3.2.1. *Expected Goals (xG)*

Expected goals (xG) is an approach commonly used for assigning value to individual shots (Green, 2012; Macdonald, 2012). General to xG approaches is the representation of the value of some shot as the probability that it results in a goal. These goal probabilities are estimated via various classification algorithms such as logistic regression, gradient boosting and deep learning methods (Mead et al., 2023). Features used in xG models typically include *action-intrinsic* variables such as distance/angle to the centre of the opponent's goal and the body part used, along with *match-context* variables such as the current match scenario (e.g. set piece, open play) (Anzer & Bauer, 2021; Mead, O'Hare, & McMenemy, 2023; Robberechts & Davis, 2020).

Cefis & Carpita (2025) incorporate spatial context into the xG model by considering the number of players ahead of the shooter (x co-ordinate greater than the shooter's) within a fixed radius (y co-ordinate $\pm 10\%$ of the shooter's). From this count, they derive two binary variables `D1.OpponentsPlayer` and `D2.OpponentsPlayer`, where the former equals 1 when the number of players in this area is greater than 0, and the latter equals 1 when the number of players in the area is greater than 1. They also consider the x and y co-ordinates of the opponent's goalkeeper (`GK x coordinate`, `GK y coordinate`). Table 3 in Cefis & Carpita (2025) show the coefficients and odds ratios of each feature in their logistic regression xG model across their 4 different techniques for addressing class imbalance. `D1.OpponentsPlayer`, `D2.OpponentsPlayer`, and `GK x coordinate` all have negative coefficients, indicating a reduction in the log odds of the shot resulting in a goal. This is particularly pronounced for `D2.OpponentsPlayer`, with a coefficient range of -6.13 to -5.69, indicating that the probability of scoring (i.e. xG) is reduced by around 99.73% (because $1 - e^{(-5.69 + -6.13)/2} \approx 0.9973$) when there is more than 1 opposition defender within proximity of the shooter. This finding matches intuitions about the influence of the opposition's positioning on goal probability and highlights how the addition of spatial context during modelling can enhance the xG metric.

3.2.2. *Expected Threat (xT)*

A further action value metric is expected threat (xT). xT measures the value of actions based on where on the pitch the action moves the ball from and to. Singh (2018) derived the following equation for calculating the xT of a given location on the pitch:

$$\mathrm{xT}_{x,y} = \left(s_{x,y} \times g_{x,y}\right)\left(m_{x,y} \times \sum_{z=1}^{Z}\sum_{w=1}^{W} T_{(x,y)\to(z,w)}\mathrm{xT}_{z,w}\right) \tag{4}$$

If we discretise the football pitch into some $Z \times W$ (Singh (2018) uses $Z = 16$ and $W = 12$) set of equally-sized square zones (x, y), we can consider three possible attacking options when the ball is in some zone (x, y): shoot, carry or pass. $\left(s_{x,y} \times g_{x,y}\right)$ quantifies the value of shooting from zone (x, y) by weighting the probability of scoring $g_{x,y}$ by the probability of shooting $s_{x,y}$. The other two options, carrying or passing, involve moving the ball from (x, y) to (z, w). The transition matrix $T_{(x,y)}$ provides the probablity of moving the ball from zone (x, y) to any zone (z, w) after 1 time step. If we consider all $Z \times W$ possible options and multiply their transition probabilities by the value of its respective (z, w) zone, we can take their sum and weight it by the probability of actually moving the ball to another zone when in (x, y). This provides a quantification of passing or carrying the ball, which when multiplied by the shot-probability weighted xG of zone (x, y) provides the xT of the ball being at zone (x, y). However, given that Equation 4 assumes that the value of each zone $\left(\mathrm{xT}_{z,w}\right)$ is already known a-priori, we must

estimate its value. The xT model thus begins by setting every zone (x, y) to have an xT of 0. This simplies Equation 4 to essentialy be the xG of zone (x, y), weighted by how likely players are to actually shoot from that zone:

$$\mathrm{xT}^{(0)}_{x,y} = \left(s_{x,y} \times g_{x,y}\right) \tag{5}$$

We can then take the xT values from this first iteration and recalculate them iteratively until convergence. Singh (2018) suggests 4–5 iterations are sufficient but that this may vary across datasets. To then assign xT to an action, we simply subtract the xT of the end location of the action (i.e. the xT of zone (z, w)) by the xT of the start location (the xT of zone (x, y)). This approach is useful for allowing us to determine the value of any on-ball attacking action, not just shots. We can also quantify genuinely valuable actions at any point during a chain of actions (e.g. an incisive pass during a sequence which ends in a goal), rather than suffering from a bias towards teminating actions (goals, assists).

Everett et al. (2022) introduce spatial context to the xT transition matrix $T_{(x,y)}$ via a pitch control (Spearman et al., 2017) and interception model (Burriel & Buldú, 2021) which was used to train a convolutional neural network (CNN) to produce the transition probabilities for their contextual transition matrix $T^{\star}_{(x,y)}$. The pitch control model considers the probability of a player controlling the ball as a function of their distance from opposition players, whilst the interception model introduces an additional transition probability, that being the probability of the ball being intercepted when moving from zone (x, y) to its intended destination (z, w). Everett, Beal, Matthews, Norman, & Ramchurn (2022) found the introduction of spatial context yielded a modest improvement in terms of model fit, reducing the log likelihood of the xT model from -5.137 to -4.151. Greater gains were found in its useful applications to tactical analysis, such as enabling the authors to apply a Basin-Hopping optimiser (Wales & Doye, 1997) to find more optimal defensive shapes than which actually occurred during actions, using their derived Threat Above Expected (TAx) metric as the algorithm's loss function.

3.2.3. *Expected Possession Value (EPV)*

Expected possession value (EPV) (Fernandez et al., 2021) is an action valuation metric, derived from other sports such as basketball (Cervone et al., 2016), which exploits tracking data to assign value to a game state (e.g. during an action) at time t via a consideration of all possible subsequent paths it can take.

$$\mathrm{EPV}_t = \sum_{a \in A} \mathbb{E}[G \mid A = a, T_t] \quad \mathbb{P}(A = a \mid T_t) \tag{6}$$

Equation 6 shows that the EPV at time t for some action is determined by the expected value of G (typically a goal being scored or not) given some action a has occurred in some spatiotemporal context T_t, multiplied by the probability of that action a occurring in the spatiotemporal context T_t. For each action a considered in the action set A, we sum these expected values to provide an EPV of some game state at time t. Thus EPV can be applied to any game state, not simply at the time at which discrete match events or actions occur.

We can see from the term T_t that EPV relies upon spatial context to operate. Fernandez, Bornn, & Cervone (2021) list a number of spatial features used in the EPV model, such as pitch control, number of players within 3m of the ball location, and the number of players within the area circumscribed by the ball and the two posts of the opponent's goal. EPV is thus inherently sensitive to spatial context in its valuation of actions. The utility provided by EPV's

incorporation of spatial context is explored by Fernandez, Bornn, & Cervone (2021). For example, they use EPV to determine the on-ball (passes with positive EPV) and off-ball (pitch locations with positive EPV if a pass were to arrive there) advantage Liverpool in 2014/15 had during possession when up against different opponent formations. This analysis shows both where Liverpool were successful and unsuccessful in their passing decision (on-ball) and also where valuable regions of the pitch are to direct passes towards during build-up (off-ball). Such an analysis is extremely useful for tactical decision making as, in response to opposition pressing structures, teams can evaluate where on the pitch they should look to progress the ball towards to be most effective during build-up play, and to evaluate how successful their current approach is.

3.2.4. *Summary*

To summarise, the discussed applications demonstrate the utility provided by incorporating spatial context into action valuation models. Given the lack of investigation into augmenting VAEP models with wider spatial context, this project addresses this research gap and explores what role representations of spatial context play in altering model behaviour and, potentially, performance.

3.3. Representing Spatial Context

3.3.1. *Manual Spatial Feature Engineering*

Previous research has represented spatial context using features engineered from spatial data.

As described in Section 3.2.1, Cefis & Carpita (2025) derived two binary features `D1.OpponentsPlayer` and `D2.OpponentsPlayer` from player x and y co-coordinates, indicating the number of players within the immediate frontal proximity of the shooter.

Peters et al. (2024) developed an Expected Pass Turnovers (xPT) metric, used to measure the probability that a given pass would result in a turnover (conceding possession to the opposition) which can be used to assess how player's passing ability during some time interval compares to what would be expected on average, given the set of passes made by that player and the contextual factors relating to those passes. The authors derive xPT values from a binary logistic mixed-effects model trained on a feature set which included two features derived from StatsBomb *three-sixty* freeze-frames: "three pressure radii" (the number of opponents within three radii of varying, but unspecified length around the ball carrier) and number of "unmarked teammates" (teammates with no opponents within 2m of them). The authors found the addition of these "positional" features improved model AUC by 6.1 points

Amichay et al. (2025) measured the areas of convex hulls and inner layers to represent team shapes. A convex hull, in this context, is the area inscribed between the outer-most positioned players on the pitch for a given team. A larger area corresponds to a wider/longer team shape, whereas a smaller area is narrower/shorter. Since this convex hull only considers the outer-most players, a second convex layer can be created from the players who do not contribute to the hull. The ratio of these two areas forms the author's derived Layer Ratio (LR) metric, which summarises the compactness of the team shape.

3.3.2. *Spatial Embeddings: Set Transformer*

An alternative way to represent spatial context is in the form of spatial embeddings. A spatial embedding is a real-valued vector used to represent a set of geometric objects (in this case, player co-ordinates and their characteristics) in a fixed d-dimensional space. These embeddings

can represent non-linear relationships between objects and thus may serve to represent patterns, like defensive and offensive team structures during actions.

However, as seen in Figure 2, StatsBomb *three-sixty* data does not identify individual players within each freeze frame (except from the actor). Thus, to create embeddings for player positions, we must use a neural network which has an input layer which is *permutation invariant*. That is, the output of the network (i.e. the embeddings) should not depend on the order in which features appear in the input layer. This would enable the locations of players in each freeze-frame to be inputted into the network in any order and still produce the same embedding. It would also be useful for such a network to attend to spatial relationships that may exist across freeze-frames, such as defensive shapes.

One useful technique for developing a permutation-invariant network for spatial embeddings is a *set transformer* (Lee et al., 2019). A set transformer is a variation on the standard transformer architecture from Vaswani et al. (2017) which can be used to process sets such that the output of the transformer is invariant to how elements in the set are permuted when entering into the model.

Transformers use multihead self-attention layers in a traditional encoder-decoder architecture. Multihead self-attention works by projecting an input sequence $X \in \mathbf{R}^{n \times d}$ into different learned linear projections of the input embeddings: $\left\{Q = XW_Q, K = XW_K, V = XW_V\right\} \in \mathbb{R}^{n \times d}$. Each row corresponds to a token representation in a different learned subspace. The self-attention mechanism determines how much each token should attend to other tokens, including itself, by computing similarity between so-called *query* vectors and *key* vectors. If the similarity between a *query* and a *key* is high, the corresponding *value* vector contributes more strongly to the output representation. This is computed for all tokens simultaneously, producing pairwise interactions between all elements in the sequence, including self-interactions. The attention mechanism achieves this via scaled dot-product attention:

$$\text{Attention } (Q, K, V) = \text{softmax } \left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{7}$$

Multihead attention then involves performing this scaled dot-product attention in parallel h times, each with their own set of Q, K, V matrices. Outputs are then concatenated and fed into a standard feedforward neural network.

The multi-head attention mechanism of standard transformers is adapted in set transformers via *induced set attention blocks* (ISABs). An ISAB performs self-attention between set elements by introducing *inducing points* which contain parameters learned during training. These inducing points are some fixed-sized vector I and attend to the inputted set X to produce a transformed set H. The input set X then attends to H to produce a final set of n elements. This method improves computational efficiency as the size of the inducing points vector $|I|$ can be set to be smaller than the input size $|X|$, thus avoiding standard pairwise self-attention and reducing time complexity from $O(n^2)$ to $O(nm)$ where $m < n$.

Important for the purposes of this present project, set transformers also enable permutation invariance by dropping position encodings of input set elements and pool outputs via pooling with multihead attention (PMA). PMA works similarly to multihead attention but instead pools the representations of each element of the set into k vectors, each providing a representation of the whole set. k is typically set to 1, provding a single representation of the set, but $k > 1$ is permitted. The queries Q are a fixed set of learned *seed vectors* S. Reordering the set will thus result in the same pooled representation via PMA, enabling permutation invariance.

As such, the encoder component of the set transformer is used to generate spatial embeddings. However, regarding the decoder, this component is simply used during the training process. The decoder takes the encoder's generated spatial embedding as an input and attempts to reconstruct the original freeze-frame array. The deviation between the original array and the decoder's array provides a means to assess how well the encoder's generated spatial embedding represents the freeze-frame array, and thereby constitutes a measure of the set transformer's performance: its loss function. The exact loss function used in this present project is discussed in Section 4.4.2.1. Given that the role of the decoder is to provide a reconstruction objective for evaluating the quality of the encoder's learned spatial embedding, it does not necessarily require the same self-attention mechanisms used by the encoder. Such a design choice to use a lightweight, non-attention based decoder has been applied in the SegFormer architecture (Xie et al., 2021) for regular transformers, and successfully applied to set transformers by Ashcroft et al. (2023) in their work on generating illustrations by training a generative set transformer model to represent them as "stroke clouds" (a type of set comprised of Bézier curves) whilst using a non-attention based diffusion decoder. In this present project, the encoder is responsible for modelling relationships between players and producing a fixed-dimensional representation of the freeze frame, while the decoder acts as a reconstruction head that maps this representation back to the original player freeze-frame. With the decoder playing an instrumental role in the development of spatial embeddings, it is arguably sufficient for the decoder to have a relatively simplistic architecture, provided the loss function used preserves permutation-invariance.

Applications of set transformers in soccer analytics have typically fallen in the domain of trajectory prediction. Kim et al. (2023) apply set transformers in the context of soccer analytics. Kim, Choi, Kim, Yoon, & Ko (2023) use set transformers to encode representations of player trajectories from tracking data snapshots to infer ball trajectories. Capellera et al. (2024) similarly developed *FootBots* using set transformer architecture to improve player and ball motion prediction over traditional models. Whilst this domain deviates from this project, these studies do highlight the appropriateness of using set transformers to encode player positions.

This project seeks to use set transformers to produced fixed-sizes spatial embeddings for each team's set of players within each freeze-frame sample. This can be achieved by first converting each individual co-ordinate pair, along with its characteristics (teammate, goalkeeper, actor), into an embedding via an initial input projection sublayer. Each set of individual co-ordinate embeddings for each team in each freeze-frame can then be passed as an input tensor to the set transformer which will produce embedding representations of each set of co-ordinates. The initial embedding of co-ordinate pairs is necessary to ensure the set transformer has enough dimensions to learn from whereas simply having 5 dimensions (i.e. x, y co-ordinates, and the 3 player attributes) would be insufficient to capture multiple non-linear relationships.

4. Methodology

4.1. Datasets Outline

The project uses data from StatsBomb's Open Data repository. This repository has "made certain leagues of StatsBomb Data freely available for public use for research projects and genuine interest in football analytics" (StatsBomb, 2026a). The StatsBomb Open Data repository provides the following datasets, and data specification documents exist which outline variables, data types and variable descriptions for each dataset:

- **competitions.json:** A single JSON file for competition summary information.
- **events:** A directory containing a single JSON file for each match, detailing each event which takes place within that match (e.g. each pass, shot, carry etc.).

- **lineups:** A directory containing a single JSON file for each match, providing information about each player who participated in that match (e.g. their position, shirt number, nationality).
- **matches:** A directory containing a single JSON file for a season in a given competition, providing summary information about each match played in that season (e.g. match date, home/away teams and scores).
- **three-sixty:** A directory containing a single JSON file for each match, providing data for each event in a given match detailing the position of each player in the broadcaster's frame at the moment of that event.

Decroos, Bransen, Van Haaren, & Davis (2019), along with VAEP, introduce the Soccer Player Action Description Language (SPADL) as a data engineering format to enable "human-interpretable, simple and complete" records of actions within football matches. A variant of this format is used in this project for the baseline feature set, that being Atomic-SPADL (Davis et al., 2020). Atomic-SPADL represents an action without considering its result (success vs failure), unlike SPADL which includes the result. An example of the effect this has on action representation is that a pass (one action under SPADL) is separated into two actions: the giving of the pass, and the reception of the pass. An additional "interception" action type is introduced to enable incomplete passes to be captured. Atomic-SPADL therefore introduces more precision in how actions are represented as actions are now indivisible (hence atomic). This atomic-SPADL format is then augmented in this project by the addition of engineered spatial features.

The project uses *events* and *three-sixty* data corresponding to 292 matches from UEFA Men's Euro 2020 & 2024, UEFA Women's Euro 2022 & 2025, FIFA Men's World Cup 2022, FIFA Women's World Cup 2023. Unlike most competitions in this repository, *three-sixty* data is available for every match in this sample, enabling player positional context to be incorporated into feature engineering for 1,052,116 match events.

4.1.1. *Limitations of Datasets*

Football analysts often makes use of tracking data (Goes, Meerhoff, Bueno, Rodrigues, Moura, Brink, Elferink-Gemser, Knobbe, Cunha, Torres, & Lemmink, 2021). Rather than considering discrete time events, tracking data provides a continuous record of player and ball position for the duration of a match. This enables analysts to model actions in matches as continuous processes, accounting for the position and movements of the entire team, unlike with events data. Such tracking data is often proprietory and is thus not available for public use (Bassek et al., 2025). This limits the scope of the present project to viewing events as discrete in time, thus losing some surrounding context which may be informative (e.g. player movement direction and velocity).

Events data is nonetheless relatively comprehensive, covering a much larger sample of matches than the limited publicly-available tracking data (Olthof & Davis, 2025). Furthermore, the use of StatsBomb's *three-sixty* data does address this limitation somewhat by providing snapshots of proximal player positions at the point of each event, allowing some inference to be made regarding team positioning during match events. However, these snapshots do not necessarily capture every player's location during an event, just those within the view of the broadcast camera (StatsBomb, 2026a).

A further limitation is with the use of SPADL formatted data. Recent work by Jo et al. (2026) has shown that some events data contains logical inconsistencies (i.e. sequences of events which are not logically possible), which the SPADL format does not correct for. The authors develop the Verified Event Data Format for Reliable Soccer Analytics (VERSA) to address this issue

using a state-transition model to automatically identify and correct logical inconsitencies, improving performance on VAEP model training. However, their results on StatsBomb data suggest that the atomic-SPADL format still slightly outperforms VERSA. This result, along with the increased precision in action representation mentioned earlier, motivates the use of the atomic-SPADL format in this project, but it is recognised that the potential data integrity issues highlighted by Jo, Kang, Lee, Lee, Bauer, & Ko (2026) will not be explicitly handeled as implementing VERSA is non-trivial and thus beyond the scope of this project.

4.2. Data Collection

Data was obtained from the StatsBomb Open Data Repository via the `statsbombpy` (StatsBomb, 2026b), `socceraction` (Socceraction Developers, n.d., Decroos, Bransen, Van Haaren, & Davis (2019)), and requests (, n.d.) libraries. The `statsbombpy` and `socceractions` libraries were used to collect raw events data from the matches in each competition, whilst `requests` was used to collect *three-sixty* freeze-frames from the StatsBomb Open Data Repository directly. Events data was then merged with freeze-frame data and each competition's data was saved to an individual parquet file.

4.3. Data Preprocessing

With StatsBomb events and *three-sixty* data collected, the first preprocessing step requires converting these events into SPADL-formatted actions. This was achieved by passing each match's set of events to the `convert_to_actions` function from `socceraction`. This also required the rejoining of freeze frame data, since this preprocessing function only returns SPADL formatted data, not custom features.

To then convert these actions to the atomic-SPADL format, the `convert_to_atomic` function similarly takes each match's set of actions and returns them in atomic-SPADL format. The atomic actions of each match from each competition are then saved to parquet files, one for each competition.

With the derived atomic-SPADL actions, spatial context features are subsequently added during data engineering.

4.4. Data Engineering

4.4.1. *Manually-Engineered Spatial Context Features*

```
{
    "teammate": boolean,
    "actor": boolean,
    "keeper": boolean,
    "location": [float, float]
}
```

Figure 2: StatsBomb *three-sixty* freeze-frame JSON schema

Figure 2 outlines the JSON schema for StatsBomb *three-sixty* freeze-frames. Each freeze-frame JSON object has an associated `event_uuid` to enable joins with events data, and a `visible_area` polygon which indicates the viewing area from the broadcast camera at the moment the action took place. We can see from the available data, that it is possible to construct the opposition proximity metric from Cefis & Carpita (2025) and a partial convex hull area or LR as explored by Amichay, Silva, Brito, & Marcelino (2025). Given that the sample in this project applies to

on-ball actions, not just shots, the opposition proximity metric from Cefis & Carpita (2025) is adapted to consider a 360° radius around the player, rather than a frontal semi-circle.

Thus, for this project, the following spatial features were engineered for teammates and opponents, respectively, using *three-sixty* data. These are both inspired by the literature and also chosen by myself:

- Euclidean distance to nearest teammate (meters)
- Euclidean distance to nearest opponent (meters)
- Number of teammates within 5 meters of the actor
- Number of opponents within 5 meters of the actor
- Average Euclidean distance to teammates in freeze-frame (meters)
- Standard deviation of Euclidean distance to teammates in freeze-frame (meters)
- Average Euclidean distance to opponents in freeze-frame (meters)
- Standard deviation of Euclidean distance to opponents in freeze-frame (meters)
- Teammate convex hull area (based on players within freeze-frame) (m^2)
- Opponent convex hull area (based on players within freeze-frame) (m^2)

To outline the rationale here, opponent spatial metrics are intended to provide a proxy measure of *pressure* applied to the actor and *defensive shape*, which imposes limitations upon where the ball can be progressed to by introducing a risk of losing possession. Teammate spatial metrics provide a proxy measure of *pass availability* for the actor and *rest defense* - the positional structure of a team whilst in possession to prepare for potential counter-attacks (Coaches' Voice, 2024).

For *number of teammates/opponents within 5 meters of the actor*, whilst a radius of 5 meters somewhat arbitrary, Bischofberger et al. (2026) argue that a "lower radius might risk missing actual pressure and involvement while a higher radius might lead to a weaker attribution between players and stronger intra-team correlations" p.5. Calabuig et al. (2024) also use a 5 meter radius around players as part of their construction of *influence areas* - the amount of overlap of two player's circles equates to how much influence the defender has over the attacker, providing a measure of defensive pressure.

It should be noted that such an approach to approximating opposition pressure or teammate pass availability is considered relatively simplistic, with more sophisticated approaches such as pitch control (Fernández et al., 2018) being preferred in the literature. However, given that metrics like pitch control require tracking data to be implemented correctly, they fall beyond this project's scope. Regardless, the intention with these manually-engineered spatial context features is to provide an approximation of spatial context, which is easy to implement, in order to contrast it with the generated spatial context embeddings in terms of classifier performance and behaviour. Given that such a measure has been successfully and usefully incorporated in previous literature, it was deemed appropriate for this present project.

4.4.2. *Spatial Context Embeddings*

To generate an n-dimensional embedding of each freeze-frame for each action, we can begin by converting each JSON object in each freeze-frame to a 1×5 array, using the 5 available features as shown in Figure 2 (the location key can be split into an x co-ordinate feature and a y co-ordinate feature, respectively). Each freeze-frame consists of n players, and so n arrays are stacked into an $n \times 5$ array.

This $n \times 5$ array, constituting each player within a single freeze-frame, is then be projected into a higher dimensional space via an initial learned affine transformation h_i.

$$h_i = W_e x_i + b_e, \quad x_i \in \mathbb{R}^5, h_i \in \mathbb{R}^{d_{\text{hidden}}} \tag{8}$$

The dimensionality d_{hidden} of this learned projection layer is left as a tunable hyperparameter during optimisation, with $d_{\text{hidden}} \in \{32, 64, 128, 256\}$ being the candidates used. This transformation provides the set transformer with a higher-dimensional latent representation of each element, enabling it to learn richer relationships between players within each freeze-frame than would be possible from the raw feature representation alone.

This projection layer $H \in \mathbb{R}^{n \times d_{\text{hidden}}}$ is then inputted to the set transformer encoder, whereby the self-attention mechanism learns the weight matrices W_Q, W_K, W_V which minimise loss (see Section 4.4.2.1).

4.4.2.1. *Set Transformer Loss Function*

Given that the freeze-frames - particularly the player co-ordinates - constitute permutation-invariant sets, the loss function used must also respect this characteristic. Each player in each freeze-frame consists of two broad components: a pair of x, y co-ordinates for their location on the pitch and a set of attributes about that player (teammate, goalkeeper, actor).

For the location component, we require a loss function which preserves the permutation-invariant property of our freeze-frames, which are unordered sets. Chamfer distance fulfills such requirements.

Chamfer distance conducts a nearest neighbour search between points in one set with points in the other set. Each point is assigned its nearest neighbour in the other set and the distance is taken between them. Each of these distances are summed and averaged across the number of elements from the first set. Minimisation of Chamfer distance is achieved by successfully reconstructing the original set. In this scenario, this would correspond to the decoder successfully reconstructing the co-ordinate locations of each player within the original freeze-frame. This nearest neighbour search can be conducted with quadratic time complexity, as we must compute every pairwise distance between original and reconstructed set elements.

$$\mathcal{L}_{\text{Chamfer}} = d_{\text{CH}}\big(S, \hat{S}\big) = \frac{1}{|S|} \sum_{x \in S} \min_{\hat{x} \in \hat{S}} \| \, x - \hat{x} \, \|_2^2 + \frac{1}{\big|\hat{S}\big|} \sum_{\hat{x} \in \hat{S}} \min_{x \in S} \| \, \hat{x} - x \, \|_2^2 \tag{9}$$

Alternative distance measures may be used in place of Chamfer distance, such as the Hungarian method (Kuhn, 1955), which can be used to find an optimal solution to the assignment problem: finding the permutation of the reconstructed set which, when paired one-to-one with the original set, minimises loss. Chamfer loss does not enforce such a one-to-one correspondence, as elements from the original set may share the same nearest neighbour from the reconstructed set (and vice versa).

Rather than searching through each permutation naively, which would result in a factorial search time complexity, the Hungarian method constructs a cost matrix which records the pairwise distances of each point in the original set with each point in the reconstructed set.

To find the optimal permutation, the minimum value of each row is subtracted from its elements. Then, the minimum value of each column is subtracted from each of its elements. If each row in the resulting matrix has an element equal to 0 where the column (j-index) of that element is not reused in the 0-valued elements of any other row, then an optimal assignment can be derived.

If this is not the case, we find the number of horizontal and vertical lines needed to cover each 0 element (a *covered value*) in the matrix. If less than n lines were used, where n is the number of rows and columns in the cost matrix, then we subtract the minimum positive-valued row value from each row element which is *uncovered* (i.e. non-zero) and add the value to elements which are covered. We then reassess whether n lines are needed to cover the 0 values and repeat until this condition is met. The resulting matrix elements with values equal to 0 correspond to the optimal one-to-one pairing of elements in the original set with elements in the reconstructed set. This approach to the Hungarian method has a cubic time complexity, resulting in a substantial improvement over the naive method.

Despite the Hungarian method having the advantage of preserving a one-to-one correspondence between original and reconstructed set elements, Chamfer loss was deemed preferable for this project as:

- Reconstructed freeze-frames are used as a training signal to produce encoder-generated embeddings which capture meaningful spatial context information. The decoder-generated freeze-frames are not needed for any downstream tasks (e.g. synthetically generating realistic football player event freeze-frames).
- Players in freeze-frames are not uniquely identifiable, so enforcing one-to-one correspondence does not necessarily provide additional semantic information.
- Chamfer loss operates with $O(n^2)$, whereas Hungarian matching is $O(n^3)$. For a training set size $n \in [10^5, 10^6]$, such as in this project, and given that the constant factor of Chamfer distance is comparatively computationally cheaper than that of the Hungarian method, the reduction of time complexity by a factor of n makes using Chamfer loss more economical.

Future work with developing spatial embeddings from freeze-frames may wish to empirically contrast alternative reconstruction loss functions whilst considering the trade-off in terms of computation time.

The second component of this set transformer's loss function relates to the teammate, actor and goalkeeper flags which each player in each freeze-frame as, as seen in Figure 2. Given that the teammate, actor and goalkeeper statuses of each player are binary categorical variables, we can model their realisations as being sampled from Bernoulli distributions. The decoder can therefore predict a probability of the flag being equal to 1 and we can use this prediction to evaluate performance.

$$\mathcal{L}_{\text{BCE}} = \frac{1}{N}\sum_{i=1}^{N}(-y_i \log \hat{p}_i - (1-y_i)\log(1-\hat{p}_i)) \tag{10}$$

Binary cross entropy (BCE) loss provides a measure of how close predicted probabilties are to the true binary values. When the true value is 1, the second term evaluates to 0 and thus the loss is determined by the first term $-y_i \log \hat{p}_i$, which evaluates to 0 when the predicted probability is 1, and approaches ∞ when the limit of the predicted probability tends to 0. When the true value is 0, the first term cancels and the second term similarly provides a measure of loss which is 0 when the predicted probability is 0, and approaches ∞ when the loss approaches 1. This loss is calculated for each data instance individually and its average is taken to provide an overall measure of BCE loss.

To combine these two measures into a single loss function for the set transformer, we can use a weighted sum, where the weights λ_{xy} and λ_f are tunable hyperparameters. Equation 11 therefore constitutes the loss function used for this present project's set transformer.

$$\mathcal{L} = \lambda_{xy}\mathcal{L}_{\text{Chamfer}} + \lambda_f \mathcal{L}_{\text{BCE}} \tag{11}$$

4.5. Set Transformer Hyperparameter Tuning

Rather than training a single set transformer with arbitrary hyperparameters, it would be preferable to use an algorithm to choose values for the model's hyperparameters as to minimise its loss function, whilst avoiding overfitting. This process is known as hyperparameter optimisation or tuning.

One standard approach to hyperparameter tuning is grid search. With grid search, a discrete set of candidate values are provided for each model hyperparameter and each possible combination of these candidates is trialled. Grid search, whilst exhaustive of the search space, is an inefficient means of hyperparameter tuning, relative to alternative methods. With h hyperparameters in the model, and v candidate values across all hyperparameters, the search space with grid search is equivalent to v^h, which constitutes an exponential time complexity $O(v^h)$. Grid search's efficacy is further compounded by the fact that much time is spent exploring hyperparameter value combinations which yield poor performance, relative to the optimal combination available. Moreover, the search space provided must be discrete, which potentially misses more optimal hyperparameter values that may be found in a continuous search space. Whilst potentially useful for small models where there exists well-established prior expectations for appropriate hyperparameter values, alternative methods may be preferred.

An alternative approach can be found in random search. Random search draws n random samples of hyperparameter value combinations from the set of values provided and trials them. Rather than an exhaustive search of a discrete set of hyperparameter value combinations, random search enables sampling over both discrete sets of values and also continuous sampling distributions for hyperparameter values. Compared to grid search, this helps to reduce the time complexity of the search space to linear time, with search time simply being a linear function of the number of desired trials - $O(n)$ - whilst also allowing for more optimal values to be chosen via the use of hyperparameter value sampling distributions. Bergstra & Bengio (2012) argue that since hyperparameters for a given model often do not contribute evenly to the performance of the model (e.g. the learning rate may be more important that the dropout rate), grid search wastes time exploring hyperparameter value combinations which will not yield significant gains in performance. Instead, random search randomly samples values for all hyperparameters at each trial, meaning that the most important hyperparameters are always being tested with new unique values within a fixed computational budget. Grid search instead wastes computational budget on varying less important hyperparameters whilst keeping important hyperparameters constant, since it must exhaust all possible combinations. Random search therefore can explore the search space of important hyperparameters more efficiently, providing opportunity for random search to outperform grid search.

Despite this advantage over grid search, a limitation common to both grid search and random search is the fact that neither approach uses information gained from previous trials to improve the estimations made by future trials. Sampling efficiency could be drastically improved if an algorithm were to consider previous trials in its sampling of new candidate values.

One family of optimiser approaches which we can use to enhance hyperparameter sampling efficiency are Bayesian optimisers. In general, a Bayesian optimiser constructs a surrogate model to predict model performance y given a set of hyperparameter values x. This surrogate model $p(y \mid x)$ produces its estimates based on some prior history of trials. After each trial, the predictions from this surrogate model are used to inform an acquisition function, which explores the search space to find regions which balance currently acquired knowledge on which hyperparameters lead to improved performance over less explored regions where uncertainty is

higher and thus potential for further optimisation may exist. This acquisition function then samples hyperparameter values based on this tradeoff, which in turn updates the surrogate model and thus the process continues for some n number of trials. A Bayesian optimisier can therefore incorporate prior evidence acquired from previous trials to inform decisions on which hyperparameter values ought to be trialled next, enabling convergence towards a local optimium.

The present project will use a type of Bayesian optimiser known as a Tree-Structured Parzen Estimator (TPE), which is outlined in the next subsection (Section 4.5.1). This TPE is implemented via the Optuna Python library (Akiba et al., 2019).

4.5.1. *Tree-Structured Parzen Estimatior (TPE)*

A TPE is a type of Bayesian optimiser which deviates from the general approach in a few key ways. Firstly, a TPE models the probability distribution of hyperparameter values, given model performance $p(x \mid y)$ - an inversion of the standard Bayesian optimiser approach. Thus, a TPE seeks to update the probability distributions of each hyperparameter in line with the prior evidence accumulated about which values lead to improved performance. $p(x \mid y)$ is defined by two derived densities: $l(x)$ and $g(x)$. Given a set of trials with candidate hyperparameters x and observed loss values y, a parameter γ is defined so as to dervied the γth quantile of y referred to as y^* such that $p(y < y^*) = \gamma$. The exact value of γ may be a fixed value (i.e. always use $\gamma = 0.15$), or as in the case with Optuna's default implementation, an adaptive gamma function:

$$\gamma(n) = \min(\lceil 0.1n \rceil, 25) \tag{12}$$

This threshold y^* is used to define two probability density estimates for the set of hyperparameters. $l(x)$ constitutes probabilities density estimates for hyperparameter values where the observed loss is less than y^*, and $g(x)$ constitutes the probability density estimates for hyperparameters values where the observed loss is greater than or equal to y^*. As such, using the definition provided by Bergstra et al. (2011) on p.4:

$$p(x \mid y) = \begin{cases} l(x) & \text{if } y < y^* \\ g(x) & \text{if } y \geq y^* \end{cases} \tag{13}$$

To then choose an optimal set of hyperparameters for the next trial, TPE aims to sample hyperparameter values which maximise Expected Improvement (EI). EI therefore constitutes the acquisition function of the TPE algorithm. EI is given as the expectation of the improvement given the surrogate model's distribution, which we can define using Bayes' rule:

$$EI(x) = \int_{-\infty}^{y^*} (y^* - y) p(y \mid x)\, dy = \int_{-\infty}^{y^*} (y^* - y) \frac{p(x \mid y) p(y)}{p(x)}\, dy \tag{14}$$

Rather than computing this expectation directly, Bergstra, Bardenet, Bengio, & Kégl (2011) instead point out since any hyperparameter combination must come from either $l(x)$ or $g(x)$, we can express $p(x)$ as:

$$p(x) = \gamma l(x) + (1 - \gamma) g(x) \tag{15}$$

We can see from Equation 14 that $EI(x) \propto \frac{1}{p(x)}$. Therefore, by substitution and then rearrangement:

$$EI(x) \propto \frac{1}{\gamma l(x) + (1-\gamma)g(x)} \propto \left(\gamma + (1-\gamma)\frac{g(x)}{l(x)}\right)^{-1} \tag{16}$$

We can therefore see that $EI(x)$ is maximised when $\frac{g(x)}{l(x)}$ is smallest, or conversely when $\frac{l(x)}{g(x)}$ is maximised. This latter ratio is used by the acquisition function. The function samples m sets of hyperparameter values from $l(x)$ and computes $\frac{l(x)}{g(x)}$. The sample with the largest value for $\frac{l(x)}{g(x)}$ is then chosen as the next candidate hyperparameter value set. This then updates the surrogate model and the process continues for n trials. In order to provide densities $l(x)$ and $g(x)$ to the acquisition function, an initial startup period of trials is required. By default, Optuna's TPE sampler uses 10 startup trials before performing the TPE algorithm for some n number of trials.

To actually estimate $l(x)$ and $g(x)$, TPE uses kernel density estimation (KDE - a.k.a. the Parzen-Rossenblatt window method) (Parzen, 1962; Rosenblatt, 1956). KDE is a non-parametric method of estimating a probability density function given some set of n observations $\{x_1, ..., x_n\}$. In the case of TPE, these observations are the candidate hyperparameters from previous trials

4.5.2. *Hyperband Pruning*

Each set of candidate hyperparameter values is trained across some number of epochs as specified during training. However, it is possible to evaluate trials more efficiently than to simply let each of then run their full course of epochs. In the context of this project, the number of epochs per trial is the resource budget. Hyperband pruning (Li et al., 2018) is a means to efficiently allocate this resource budget across trials as to avoid wasting training epochs on trials which are clearly performing poorly.

Firstly, hyperband tuning involves three key parameters: R_{min}, R_{max} and η. R_{min} is the minimum resource to be used per trial. As mentioned, for this project this is the minimum number of epochs each trial will train with. R_{max} is the resource budget we are choosing to employ (i.e. the maximum number of epoch per trial). η is known as the reduction factor.

Hyperband tuning uses the the successive halving algorithm to assess the performance of a trial during its progress. Whilst the standard successive halving algorithm involves running trials in parallel, the implementation in Optuna involves running trial in succession and thus an alternative method is employed. For each trial, we evaluate performance after the first rung is completed (i.e. R_{min}) and compare it to the performance of previous trials at that same rung. If performance is competitive with previous trials, we continue to the next rung, which is the current rung multiplied by the reduction factor η. We continue until either performance fails to compete, in which case the trial is terminated (i.e. *pruned*), or we reach the resource budget R_{max}.

What separates hyperband tuning from standard successive halving is that different sets of rungs are employed for some trials (known as brackets). Some brackets may involve starting with a smaller amount of resource, corresponding to more exploratory behaviour, whilst others may adopt a greater amount of resource initially, corresponding to a more exploitative approach. Trials are only compared to previous trials of the same bracket. This approach allows hyperband tuning to benefit from a balance of both exploratory or exploitative approaches, whereas standard successive halving must commit to one or the other. The number of brackets s_{max} used during tuning is determined as follows:

$$s_{\max} = \left\lfloor \log_{\eta}\left(\frac{R_{\max}}{R_{\min}}\right) \right\rfloor \tag{17}$$

4.6. Classifier Training

Decroos, Bransen, Van Haaren, & Davis (2019) use 4 different supervised classification techniques to estimate scoring and conceding probabilities, enabling VAEP calculations. This project will replicate the implementation of these same 4 techniques as closely as possible, albeit they do not provide comprehensive details on the Python version used, librarie versions, or all model hyperparameters.

Given that VAEP requires two concurrent classifiers to be trained for scoring and conceding probability estimates, respectively, and the introduction of manually engineered spatial features and spatial embeddings will result in a total of 32 individual classifier models being trained, enablining the construction of: baseline VAEP; baseline + manual features; baseline + embeddings; and baseline + manual features + embeddings models. This ablation design allows for the impact of adding spatial context on model performance and behaviour to be investigated.

4.6.1. *Logistic Regression*

Logistic regression (Cox, 1958) is a classification technique which estimates the probability that a given data instance belongs to some reference class (i.e. an action results in a goal scored/ conceded vs no goal scored/conceded, within k events). Logistic regression learns a vector of weights $\mathbf{w}$ and bias b which minimises cross-entropy loss and produces a probability estimate for the data instance beloning to the reference class:

$$P(Y = 1 \mid X) = \frac{1}{1 + e^{-(\mathbf{w}^{\top}\mathbf{x}+b)}} \tag{18}$$

Decroos, Bransen, Van Haaren, & Davis (2019) report the use of L2 Regularisation and a Limited-memory BFGS, which is replicated in this project.

L2 regularisation introduces an additional penalty term to the loss function (in this case cross-entropy loss, see Section 4.6.4). This penality term is some multiple $\lambda \in \mathbb{R}^{+}$ of the sum of squared weights:

$$\text{L2} = \lambda \sum_{i=1}^{N} w_i^2 \tag{19}$$

`scikit-learn` uses a hyperparameter $C = \frac{1}{\lambda}$ to control regularisation strength, which is set to 1 by default. Since the authors do not specify otherwise, this default will be maintained for training.

Limited-memory BFGS (Liu & Nocedal, 1989) is a computationally-efficient optimisation method, used in logistic regression to iteratively minimise the loss function during training. Again, this hyperparameter is the default for `scikit-learn`.

4.6.2. *Random Forest*

Classification tasks may also be performed by decision trees. We can treat a classification task as a form of recursive partitioning of the dataset until we obtain maximally *pure* partitions primarily consisting of data instances from a single classification class. Choosing a feature and

subsequent splitting threshold which minimises impurity (see Section 4.6.5) until a maximum tree depth or minimum leaf node size is reached builds a decision tree which can be used to make classifications by applying a sequence of logical filters.

A random forest (Breiman, 2001) is an ensemble method which aggregates results from multiple decision trees, each trained on a random subset of the training set. In a classification task, this aggregation is employed by majority voting. For each data instance, the class predictions made by each tree in the forest are counted up and the class with the most "votes" is outputted as the predicted class for that instance. By having class predictions be made via majority voting from multiple decision trees, a random forest benefits from being less prone to overfitting and more robust to outliers in the dataset.

Decroos, Bransen, Van Haaren, & Davis (2019) indicate that they used 100 trees and 40 parallel threads to train their random forest models. Thus this project also used 100 trees. However, the number of parallel threads simply determines the number of concurrent processes used, which does not impact model behaviour other than via its runtime. Since my hardware is different to the authors', this hyperparameter will be chosen during training and reported in the write up.

4.6.3. *Gradient Boosting*

Gradient boosting (Friedman, 2001) is another ensemble method involving decision trees. Gradient boosting methods construct a "strong" predictor by sequentially adding "weak" learners, where each new learner is trained to correct the errors of the previous ensemble.

$$\hat{y}_i = \sum_{t=1}^{T} \eta f_t(x_i), \quad f_t \in \mathcal{F} \tag{20}$$

Each function f_t is learned sequentially to minimise a loss function (like cross-entropy loss). At each iteration, the new weak learner is fit to the negative gradient of the loss with respect to the current model predictions. The learning rate η is used to determine how much the next tree contributes to model estimates.

4.6.3.1. *XGBoost*

XGBoost (Chen & Guestrin, 2016) builds upon standard gradient boosting by incorporating regularisation and introducing improved computational efficiency, even being capable of handling sparse data (i.e. data instances which are empty in some subset of features). Decroos, Bransen, Van Haaren, & Davis (2019) use 100 trees which are allowed to reach a maximum depth of 3 nodes. The learning rate η is equal to 0.1. Again, 40 parallel threads are used. Each of these hyperparameters (except the parallel threads) will be held constant in this project.

4.6.3.2. *CatBoost*

CatBoost (Prokhorenkova et al., 2019) is a gradient boosting method which is optimised for processing categorical predictors without requiring manual one-hot encoding. Decroos, Bransen, Van Haaren, & Davis (2019) retain the default hyperparameters for CatBoost (CatBoost Developers, n.d.), and again use 40 parallel threads.

4.6.4. *Loss Function: Cross-Entropy Loss*

Logistic regression, XGBoost and CatBoost each minimise cross-entropy loss in their classification predictions, according to their default implementations in the aforementioned Python libraries. Cross-entropy measures how "surprised" the model is by observations drawn

from the true data-generating distribution $p(x)$, when evaluated under the predicted distribution $q(x)$. If the model tends to assign low probability to data instances which are probable under $p(x)$, then cross-entropy will be high. A low cross-entropy is found when $q(x)$ approximates $p(x)$ well.

$$H(p, q) = -\sum_{x \in X} p(x) \log q(x) \tag{21}$$

4.6.5. *Split Criterion: Gini impurity*

Whilst the split criterion used for the random forest models is not specified, the authors do indicate that they "used the implementation from the `scikit-learn` Python package" (Decroos, Bransen, Van Haaren, & Davis, 2019, p. 1860). It is thus inferred that they use the default split criterion of Gini impurity (Pedregosa et al., 2011; scikit-learn Developers, n.d.).

Gini impurity indicates the probability of assigning an incorrect label to a randomly chosen data instance by simply using the relative frequence of each label's occurrence in the data.

$$G_m = 1 - \sum_{k=1}^{K} \left(\frac{n_{m_k}}{n_m} \right)^2 \tag{22}$$

Decision trees within random forest classifiers, at each splitting decision, choose the feature and value threshold s which minimises the sum of the weighted Gini impurities of each partition m:

$$G_{m_{<s}/m_{>s}} = \frac{n_{m_{<s}}}{n_m} G_{m_{<s}} + \frac{n_{m_{>s}}}{n_m} G_{m_{>s}} \tag{23}$$

This results in the *purest* possible split and thus corresponds to the most discriminative splitting decision available. This process then iterates on each node in the decision tree until leaf nodes are reached which each only contain a single class (in this case, 0 or 1). Data instances can then be classified according to this derived decision tree.

4.7. Model Evaluation Metrics

4.7.1. *Area Under the Receiver Operating Characteristic Curve (AUC-ROC)*

A receiver operating characteristic curve (ROC) is produced by plotting the true-positive rate (TPR) against the false positive rate (FPR) for each probability threshold a model could use to classify a data instance into the reference class (i.e. the action results in a goal scored/conceded within k events).

4.7.2. *Brier Score*

Decroos, Bransen, Van Haaren, & Davis (2019) use the Brier score (Brier, 1950) to measure the performance of the VAEP classifier models by comparing the model's predicted probability of a data instance belonging to a class with its true class membership.

$$\text{BS} = \frac{1}{N} \sum_{i=1}^{N} \sum_{k=1}^{K} (p_{ik} - y_{ik})^2 \tag{24}$$

This is effectively equivalent to the mean squared error, where predictions equate to probabilities of belonging to the kth class and the true values are a one-hot encoding of that data instance's membership of that k^{th} class. For binary classification this simplifies to:

$$\mathrm{BS}_{\mathrm{binary}} = \frac{1}{N}\sum_{i=1}^{N}(p_i - y_i)^2 \tag{25}$$

A lower Brier score indicates that the model's predicted probabilities are, on average, closer to the true outcomes.

4.7.3. *Cross-Entropy Loss*

Cross-entropy loss was introduced as a loss function in Section 4.6.4, but it may also be used as a classification model evaluation metric. This project intends to use cross-entropy loss to evaluate each of the two classifier models which constitute each VAEP model.

4.7.4. *Expected Calibration Error (ECE)*

Expected calibration error (ECE) (Naeini et al., 2015) is a performance metric used to evaluate the ability of a classification model to make accurate predictions. ECE creates m bins between 0 and 1 and assigns predictions to bins based on their probability estimates. For each bin, we take the accuracy of predictions in that bin o_i and subtract from it the average of the probability estimates in that bin e_i. This is weighted by the proportion of predictions in that bin. Each bin's score is the summed to provide the ECE of the model's predictions. A lower ECE is more desirable, indicating that model predictions are close to true frequencies.

$$\mathrm{ECE} = \sum_{i=1}^{K} P(i) \cdot |o_i - e_i|$$

4.8. Model Interpretability

4.8.1. *Permutation Importance*

Permutation importance is a method for estimating feature importance by randomly permuting one or more features and evaluating the change in model performance metrics. Features which, when permuted, result in a larger drop in model performance can be inferred to be more important in model estimations. Care should be taken with multicolinearity, as permutations will not entirely disconnect their relationship with others model features since those relationships can continue to exist via their colinear counterparts.

Permutation importance will thus be used in this project to highlight which features or set of features are most determinative of the VAEP values assigned to actions. The potential prescence of multicolinarity could be address via hierarchical clustering

4.8.2. *Partial Dependence Plots (PDPs)*

Partial dependence plots (PDPs) (Friedman, 2001) are used to visualise the relationship between some set of features (usually just 1 or 2) and the predictions made by a machine learning model, holding other features constant. In this case, we can use PDP plots to explore our interpretable feature set, that is the baseline and manually-engineered features, as opposed to the spatial embeddings whose dimensions are not directly interpretable.

4.8.3. *k-Means Clustering*

We can also explore the spatial embeddings via k-means clustering, similarly to how DeepCluster applies k-means to image embeddings generated by a convolutional neural network (CNN) (Caron et al., 2019). k-means is an unsupervised clustering algorithm which finds optimal locations for some k centroids within the d-dimensional feature space. The algorithm initialises k centroids within the feature space, assigns data points to their nearest centroid and then recalculates the location of each centroid to the position which minimises the distance to its assigned data points. The data points are then reassigned and the algorithm iterates until convergence. A number of distance measures may be used, with euclidean distance being most common.

$$\text{dist } (x_i, x_n) = \sqrt{\sum_{d=1}^{D} \left(x_{i_d} - x_{n_d} \right)^2} \tag{26}$$

The benefit of using k-means clustering with the present project's spatial embeddings is that underlying relationships represented by these embeddings can be explored by comparing the properties of clusters. For instance, player position heatmaps for each cluster could be plotted to potentially reveal notable team shapes, or if certain clusters tend to involve certain action types over others.

4.8.3.1. *Choosing* k

By fitting a model to the data with varying values of k, we can use a number of metrics to evaluate how appropriate the chosen value of k is for clustering the given data points.

4.8.3.1.1. *Inertia*

The hyperparameter k can be chosen by considering the marginal reduction in *inertia* as k increases. A suitable k can be chosen at the point where this marginal reduction becomes negligible (this is known as the *elbow method*):

$$\text{Inertia} = \sum_{j=1}^{K} \sum_{x_i \in C_j} \parallel x_i - \mu_j \parallel^2 \tag{27}$$

Whilst commonly used in deriving k and a somewhat informative heuristic, the elbow method is inherently statistically arbitrary as a means for determining the appropriate number of clusters (Schubert, 2023).

4.8.3.1.2. *Average Silhouette Score*

An additional metric is the average *silhouette score* (Rousseeuw, 1987) for each value of k. For each data instance i, we can determine its average distance to others instances in the same cluster (known as *cohesion* $a(i)$) and its average distance to instances in the next nearest cluster (known as *separation* $b(i)$). Taking the differences between the separation and cohesion and dividing by the maximum of the two, we get the data instance's silhouette score.

If separation is lower than cohesion for a data instance, the silhouette score will be negative and thus suggests the data instance is closer to some other cluster(s) relative to its distance to its assigned cluster. Likewise, greater cohesion over separation suggests the data instance is appropriately assigned to its cluster, in terms of the distance metric used.

An average can then be taken across all data points and this average score can be compared across different values for k, where 1 indicates good overall clustering and -1 poor:

$$\text{SC} = \frac{1}{N}\sum_{i=1}^{N}\frac{b(i)-a(i)}{\max\{a(i),b(i)\}} \tag{28}$$

4.8.3.1.3. *Calinski-Harabasz Index (CHI)*

We may also appeal to the Calinski-Harabasz index (CHI) (Caliński & Harabasz, 1974) in assessing a suitable k. CHI is just a ratio of the between-cluster sum of squares (BCSS) and the within-cluster sum of squares (WCSS, i.e. the inertia), where the former is the sum of squared distances between cluster centroids and the latter is the sum of squared distances of each data point to its respective cluster. Each of these sum of squares measures are normalised by their respective degrees of freedom.

$$\text{CHI} = \frac{\frac{1}{k-1}\sum_{i=1}^{k} n_i \parallel c_i - c \parallel^2}{\frac{1}{n-k}\sum_{i=1}^{k}\sum_{x\in C_i}\parallel x - c_i \parallel^2} \tag{29}$$

4.8.3.1.4. *Davies-Bouldin Index (DBI)*

Lastly, the Davies-Bouldin index (DBI) provides a simultaneous evaluation of how separated each cluster is and how compact data instances are to their assigned cluster. For each pair of clusters i, j, we can consider how their individual compactness ($S_i and S_j$) and their relative distances compare by taking the ratio of the sum of their compactness to the distance of their centroids.

$$R_{ij} = \frac{S_i + S_j}{M_{ij}} \tag{30}$$

S_i and M_{ij} can, in principle, be any measure of internal compactness or cluster separation, respectively. `scikit-learn` implements the average Euclidean distance between data instances and their cluster's centroid for internal compactness, and Euclidean distance between cluster centroids for cluster separation.

A lower value for R_{ij} indicates the clusters i, j are well-separated and internally compact. As such, for each cluster i, we can consider its *worst-case scenario* by finding its corresponding j^{th} cluster which maximises R_{ij}. Taking the sum of each of these maximum R_{ij} values and dividing by the number of clusters k gives a measure of the average clustering performance $\bar{R}$, where minimising the measure optimises clustering performance:

$$\bar{R} = \frac{1}{k}\sum_{i=1}^{k}\max_{i\neq j} R_{ij} \tag{31}$$

4.8.4. *Uniform Manifold Approximation and Projection (UMAP)*

In order to interpret the spatial embeddings, uniform manifold approximation and projection (UMAP) (McInnes et al., 2020) can be used as a dimensionality reduction technique to represent embeddings in a 2 or 3 dimensional space, which can then be visualised. The color of data points within a UMAP can then be determined by properties such as the VAEP value, scoring/conceding probability or k-means clusters.

5. Results

5.1. Set Transformer Training

The set transformer used to generate spatial embeddings was trained on 652,041 atomic-spadl actions, with a validation set of 95,933 used for hyperparameter tuning. A final test set of 191,180 instances was used to test the best performing model following hyperparameter tuning. This best performing model was trained for 20 epochs, whereas models with candidate hyperparameter during tuning were trained for 10 epochs, unless pruned by the hyperband pruner.

A total of 30 trials were used for hyperparameter tuning. Table 1 shows the set of candidate values which the TPE sampler drew from. As indicated by the `Data Type` column, most hyperparameters were passed as categorical sets, whilst those which were continuous allowed the TPE sampler to sample from any value within the specified range, inclusive. The learning rate and weight decay were additionally specified to be sampled log-uniformly.

Hyperparameter	Candidate Values	Data Type
Batch Size	[256, 512, 1024]	Categorical
Dropout	[0.0, 0.1, 0.2, 0.3]	Categorical
Embedding Dimensions	[16, 32, 64]	Categorical
Flag Weight	(0.05, 0.5)	Continuous
Hidden Dimensions	[32, 64, 128, 256]	Categorical
Learning Rate	(0.0001, 0.005)	Continuous (Logarithmic)
Number of Heads	[2, 4, 8]	Categorical
Number of Hidden Layers	[1, 2, 3, 4]	Categorical
Weight Decay	(1e-06, 0.001)	Continuous (Logarithmic)
XY Weight	(0.5, 2.0)	Continuous

Table 1: Hyperparameter Candidate Values

As seen in Table 2, trial 22 performed best in terms of minimising the weighted Chamfer + BCE loss function.

trial	value	batch size	output dim	flag weight	hidden dim	drop -out	lr	num heads	num layers	weight decay	xy weight
0	60.875	256	16	0.204	32	0.3	1.8e-03	8	3	6.8e-05	1.58
1	1437.236	512	32	0.191	128	0.1	1.1e-03	8	4	3.6e-05	1.42
2	18.687	512	32	0.351	256	0.0	1.4e-04	2	3	1.5e-04	0.52
3	518.686	256	64	0.429	256	0.1	1.6e-03	2	2	5.3e-06	1.02
4	nan	512	32	0.193	128	0.3	8.7e-04	2	1	3.6e-05	1.85
5	48.240	256	64	0.086	256	0.0	1.4e-04	4	2	1.3e-04	1.76
6	nan	512	16	0.247	32	0.1	2.4e-03	4	4	2.9e-04	1.86
7	52.487	1024	32	0.414	128	0.2	2.2e-04	2	4	2.6e-05	0.68
8	100.873	1024	16	0.407	128	0.0	6.6e-04	8	3	5.6e-04	1.88
9	107.329	1024	64	0.069	64	0.0	1.0e-04	4	3	2.5e-04	1.70
10	121.714	512	32	0.314	256	0.2	4.4e-03	2	1	1.4e-06	0.52
11	58.463	256	16	0.319	32	0.3	3.9e-04	8	3	7.7e-05	1.13
12	50.449	256	16	0.488	32	0.3	3.7e-04	8	3	8.8e-04	1.00
13	117.604	256	32	0.484	64	0.3	2.7e-04	2	3	8.5e-04	0.86
14	65.936	512	16	0.488	32	0.0	3.8e-04	8	3	3.5e-04	1.26
15	nan	256	16	0.344	256	0.3	1.8e-04	2	3	9.4e-06	0.80
16	16.950	512	32	0.372	256	0.0	4.6e-04	8	3	9.6e-04	0.60
17	26.969	512	32	0.369	256	0.0	1.0e-04	2	2	1.5e-04	0.53
18	18.106	512	32	0.284	256	0.0	5.4e-04	8	1	4.4e-04	0.69
19	25.044	512	32	0.264	256	0.0	4.9e-04	8	1	4.4e-04	0.73
20	29.878	512	32	0.281	256	0.0	6.8e-04	8	1	9.5e-04	0.93
21	28.831	512	32	0.367	256	0.0	2.5e-04	8	1	1.1e-04	0.64
22	14.629	512	32	0.297	256	0.0	5.8e-04	8	1	2.2e-04	0.59
23	32.188	512	32	0.230	256	0.0	5.7e-04	8	1	5.1e-04	0.68
24	39.550	512	32	0.149	64	0.2	9.5e-04	8	1	2.5e-04	0.83
25	15.982	512	32	0.296	256	0.0	4.6e-04	8	1	5.2e-04	0.65
26	36.474	1024	64	0.389	256	0.0	3.5e-04	8	1	2.0e-04	0.60
27	32.159	512	32	0.313	256	0.0	8.4e-04	4	1	6.2e-04	0.79
28	577.291	512	32	0.450	256	0.0	1.3e-03	8	4	6.8e-05	0.92
29	57.894	512	32	0.229	64	0.2	3.1e-04	8	2	3.4e-04	1.09

Table 2: Optuna Hyperparameter Tuning Results

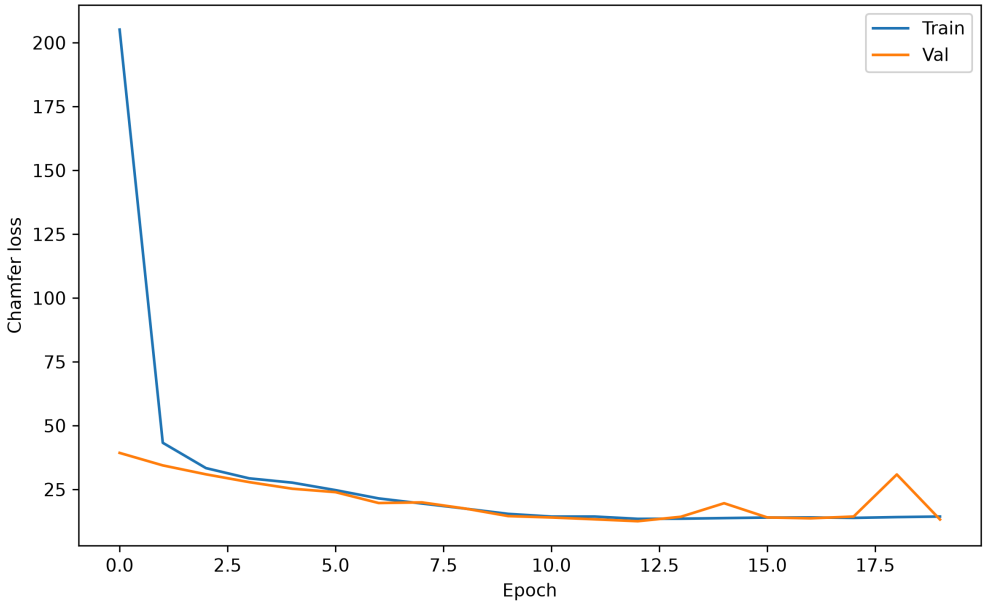


Figure 3: Set Transformer Training Loss Curve

5.2. Model Training

model	feature set	target	brier score	log loss	ece	auc
CatBoost	all	concedes	0.001125	0.007327	0.000477	0.887021
CatBoost	baseline	concedes	0.001167	0.007428	0.000423	0.890564
CatBoost	embeddings	concedes	0.001142	0.007412	0.000451	0.890103
CatBoost	manual	concedes	0.001173	0.007635	0.000473	0.883410
LR	all	concedes	0.001175	0.007585	0.000181	0.885107
LR	baseline	concedes	0.001192	0.007853	0.000101	0.872270
LR	embeddings	concedes	0.001177	0.007640	0.000088	0.881908
LR	manual	concedes	0.001188	0.007694	0.000141	0.877531
RF	all	concedes	0.001239	0.008040	0.000227	0.894979
RF	baseline	concedes	0.001240	0.008171	0.000241	0.887575
RF	embeddings	concedes	0.001228	0.008096	0.000192	0.882123
RF	manual	concedes	0.001234	0.008056	0.000272	0.894220
XGBoost	all	concedes	0.001147	0.006937	0.000145	0.913801
XGBoost	baseline	concedes	0.001179	0.007199	0.000120	0.897184
XGBoost	embeddings	concedes	0.001143	0.006871	0.000110	0.917760
XGBoost	manual	concedes	0.001156	0.006968	0.000196	0.906989

Table 3: Concedes Classifiers Test Results

model	feature set	target	brier score	log loss	ece	auc
CatBoost	all	scores	0.005960	0.032472	0.001515	0.838349
CatBoost	baseline	scores	0.005996	0.033541	0.001110	0.807787
CatBoost	embeddings	scores	0.005960	0.032670	0.001455	0.834615
CatBoost	manual	scores	0.006009	0.033148	0.001314	0.824931
LR	all	scores	0.005905	0.031800	0.000799	0.853255

model	feature set	target	brier score	log loss	ece	auc
LR	baseline	scores	0.005955	0.033980	0.000781	0.772612
LR	embeddings	scores	0.005900	0.031748	0.000936	0.854696
LR	manual	scores	0.005942	0.032962	0.000694	0.816833
RF	all	scores	0.006463	0.035154	0.002268	0.805724
RF	baseline	scores	0.006472	0.037071	0.002074	0.755487
RF	embeddings	scores	0.006380	0.035205	0.002037	0.799344
RF	manual	scores	0.006514	0.036883	0.002111	0.756424
XGBoost	all	scores	0.005869	0.031383	0.000787	0.856037
XGBoost	baseline	scores	0.005881	0.032361	0.000785	0.821213
XGBoost	embeddings	scores	0.005857	0.031320	0.000849	0.855174
XGBoost	manual	scores	0.005879	0.031918	0.000718	0.839078

Table 4: Scores Classifiers Test Results

5.2.1. *Feature Set Significance Tests*

5.2.1.1. *Concedes Classifier*

comparison	z	p holm	delta auc	auc 1	auc 2
all vs baseline	0.53	1	−3.54e-03	0.8870	0.8906
all vs embeddings	0.63	1	−3.08e-03	0.8870	0.8901
all vs manual	−0.65	1	3.61e-03	0.8870	0.8834
baseline vs embeddings	−0.08	1	4.61e-04	0.8906	0.8901
baseline vs manual	−1.25	1	7.15e-03	0.8906	0.8834
embeddings vs manual	−1.07	1	6.69e-03	0.8901	0.8834

Table 5: Concedes Model Feature Set Significance Test Comparison (CatBoost)

comparison	z	p holm	delta auc	auc 1	auc 2
all vs baseline	−1.57	0.7	1.28e-02	0.8851	0.8723
all vs embeddings	−1.34	0.9	3.20e-03	0.8851	0.8819
all vs manual	−1.25	0.9	7.58e-03	0.8851	0.8775
baseline vs embeddings	1.14	0.9	−9.64e-03	0.8723	0.8819
baseline vs manual	1.04	0.9	−5.26e-03	0.8723	0.8775
embeddings vs manual	−0.67	0.9	4.38e-03	0.8819	0.8775

Table 6: Concedes Model Feature Set Significance Test Comparison (Logistic Regression)

comparison	z	p holm	delta auc	auc 1	auc 2
all vs baseline	−0.93	1	7.40e-03	0.8950	0.8876
all vs embeddings	−3.25	0.007	1.29e-02	0.8950	0.8821
all vs manual	−0.14	1	7.59e-04	0.8950	0.8942
baseline vs embeddings	−0.64	1	5.45e-03	0.8876	0.8821
baseline vs manual	0.97	1	−6.64e-03	0.8876	0.8942
embeddings vs manual	1.75	0.4	−1.21e-02	0.8821	0.8942

Table 7: Concedes Model Feature Set Significance Test Comparison (Random Forest)

comparison	z	p holm	delta auc	auc 1	auc 2
all vs baseline	−2.90	0.02	1.66e-02	0.9138	0.8972
all vs embeddings	1.76	0.2	−3.96e-03	0.9138	0.9178
all vs manual	−1.37	0.2	6.81e-03	0.9138	0.9070
baseline vs embeddings	3.75	0.001	−2.06e-02	0.8972	0.9178
baseline vs manual	2.18	0.1	−9.81e-03	0.8972	0.9070
embeddings vs manual	−2.20	0.1	1.08e-02	0.9178	0.9070

Table 8: Concedes Model Feature Set Significance Test Comparison (XGBoost)

5.2.1.2. *Scores Classifier*

comparison	z	p holm	delta auc	auc 1	auc 2
all vs baseline	−6.31	2e-09	3.06e-02	0.8383	0.8078
all vs embeddings	−1.17	0.2	3.73e-03	0.8383	0.8346
all vs manual	−3.16	0.005	1.34e-02	0.8383	0.8249
baseline vs embeddings	5.84	3e-08	−2.68e-02	0.8078	0.8346
baseline vs manual	4.40	4e-05	−1.71e-02	0.8078	0.8249
embeddings vs manual	−2.19	0.06	9.68e-03	0.8346	0.8249

Table 9: Scores Model Feature Set Significance Test Comparison (CatBoost)

comparison	z	p holm	delta auc	auc 1	auc 2
all vs baseline	−13.42	2e-40	8.06e-02	0.8533	0.7726
all vs embeddings	1.13	0.3	−1.44e-03	0.8533	0.8547
all vs manual	−10.87	6e-27	3.64e-02	0.8533	0.8168
baseline vs embeddings	13.97	1e-43	−8.21e-02	0.7726	0.8547
baseline vs manual	8.41	8e-17	−4.42e-02	0.7726	0.8168
embeddings vs manual	−10.03	3e-23	3.79e-02	0.8547	0.8168

Table 10: Scores Model Feature Set Significance Test Comparison (Logistic Regression)

comparison	z	p holm	delta auc	auc 1	auc 2
all vs baseline	−6.58	3e-10	5.02e-02	0.8057	0.7555
all vs embeddings	−3.88	0.0002	6.38e-03	0.8057	0.7993
all vs manual	−6.33	1e-09	4.93e-02	0.8057	0.7564
baseline vs embeddings	5.74	4e-08	−4.39e-02	0.7555	0.7993
baseline vs manual	0.19	0.9	−9.36e-04	0.7555	0.7564
embeddings vs manual	−5.27	4e-07	4.29e-02	0.7993	0.7564

Table 11: Scores Model Feature Set Significance Test Comparison (Random Forest)

comparison	z	p holm	delta auc	auc 1	auc 2
all vs baseline	−8.80	7e-18	3.48e-02	0.8560	0.8212
all vs embeddings	−0.69	0.5	8.63e-04	0.8560	0.8552
all vs manual	−5.26	4e-07	1.70e-02	0.8560	0.8391
baseline vs embeddings	8.84	6e-18	−3.40e-02	0.8212	0.8552
baseline vs manual	6.03	7e-09	−1.79e-02	0.8212	0.8391
embeddings vs manual	−4.73	5e-06	1.61e-02	0.8552	0.8391

Table 12: Scores Model Feature Set Significance Test Comparison (XGBoost)

5.2.2. *VAEP EDA*

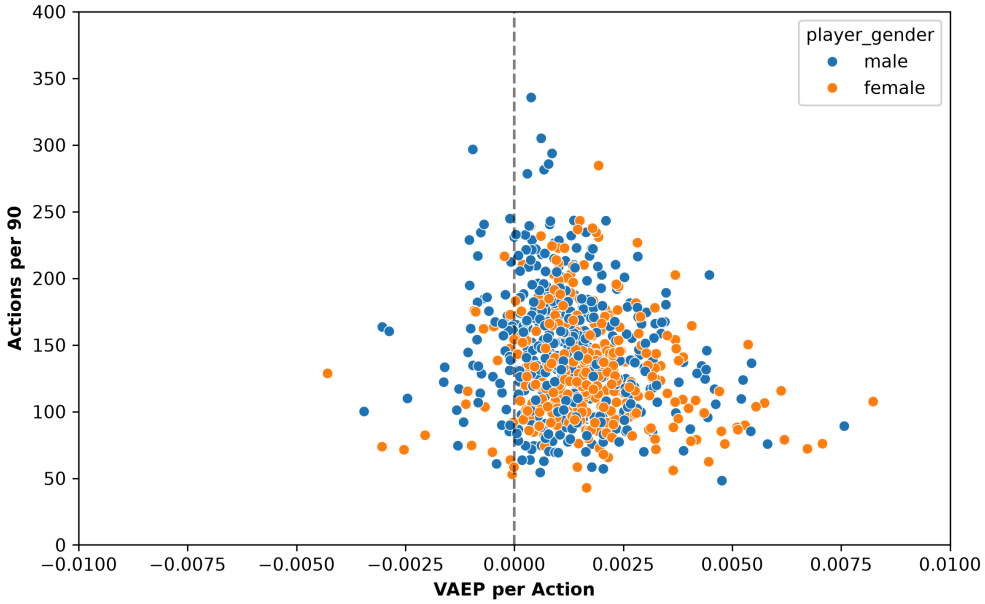


Figure 4: VAEP per Action vs Actions per 90 Minutes Played

5.3. Model Interpretation

5.3.1. *Permutation Importance*

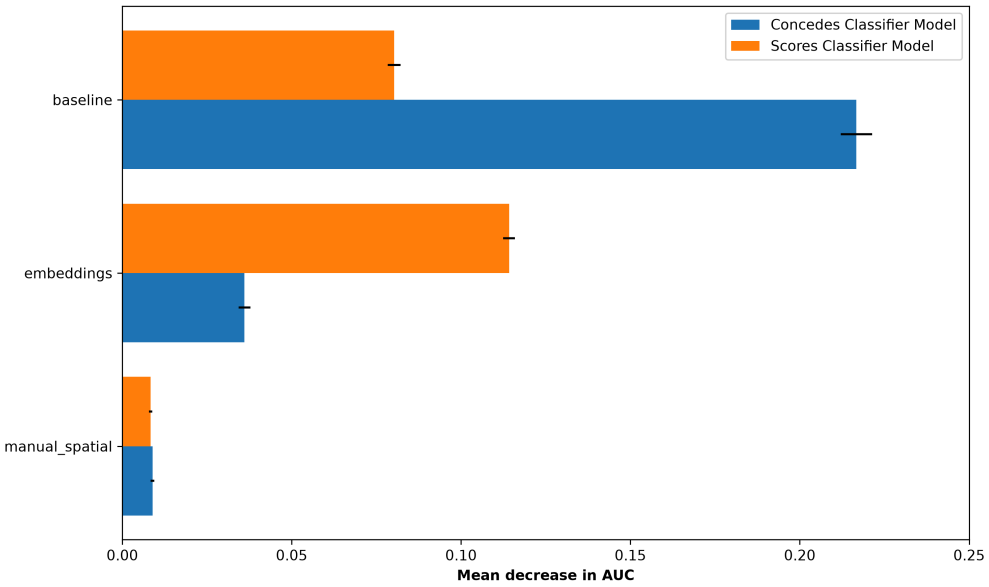


Figure 5: Grouped Permutation Importance

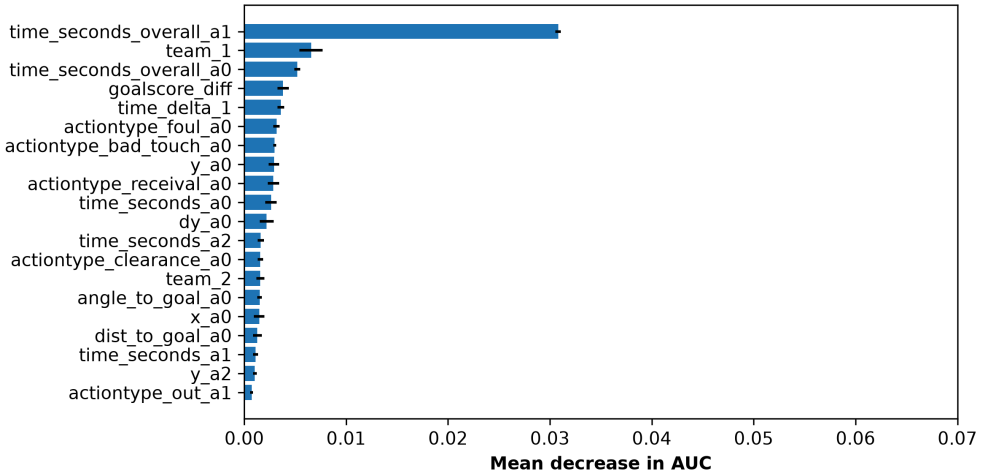


Figure 6: Concede Classifier Permutation Importance: Baseline Features

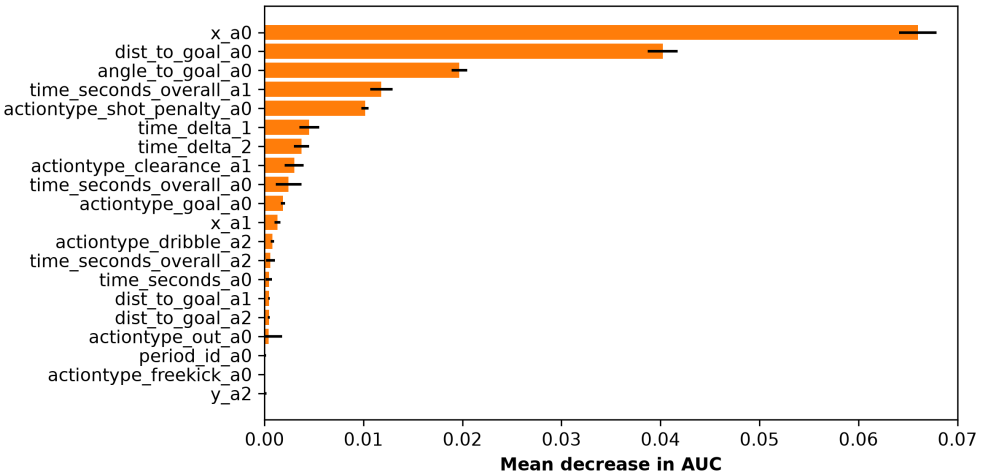


Figure 7: Score Classifier Permutation Importance: Baseline Features

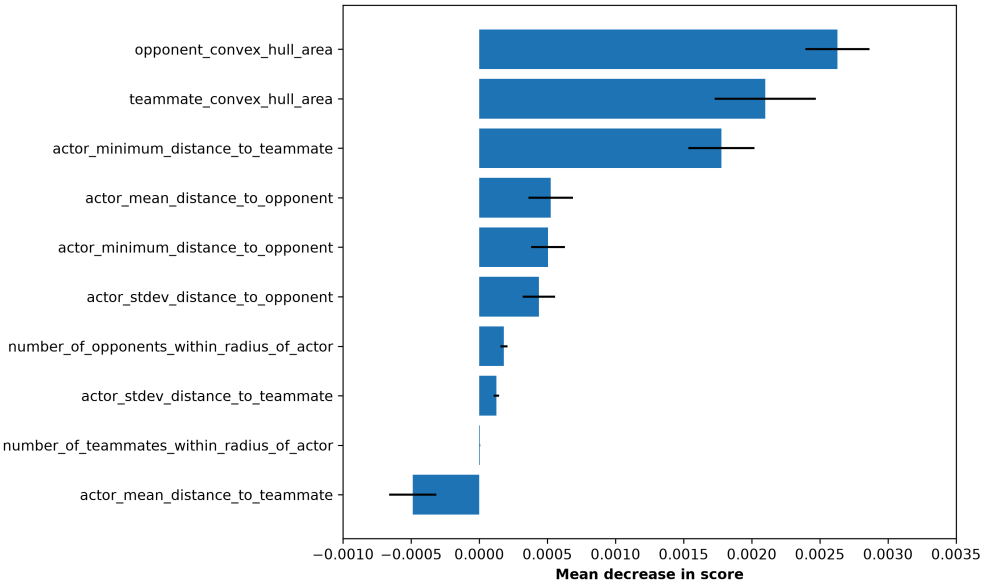


Figure 8: Concede Classifier Permutation Importance: Manually Engineered Spatial Features

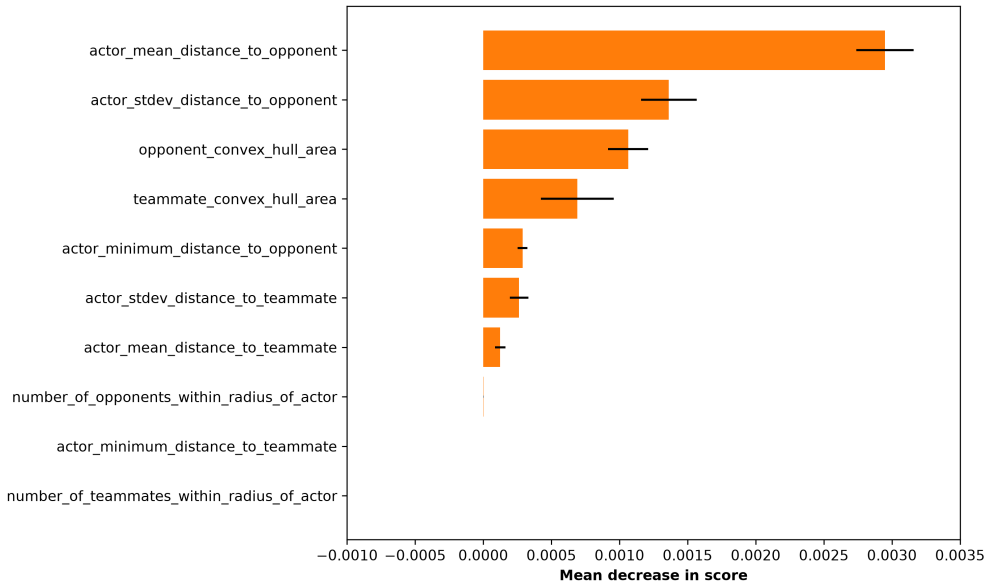


Figure 9: Score Classifier Permutation Importance: Manually Engineered Spatial Features

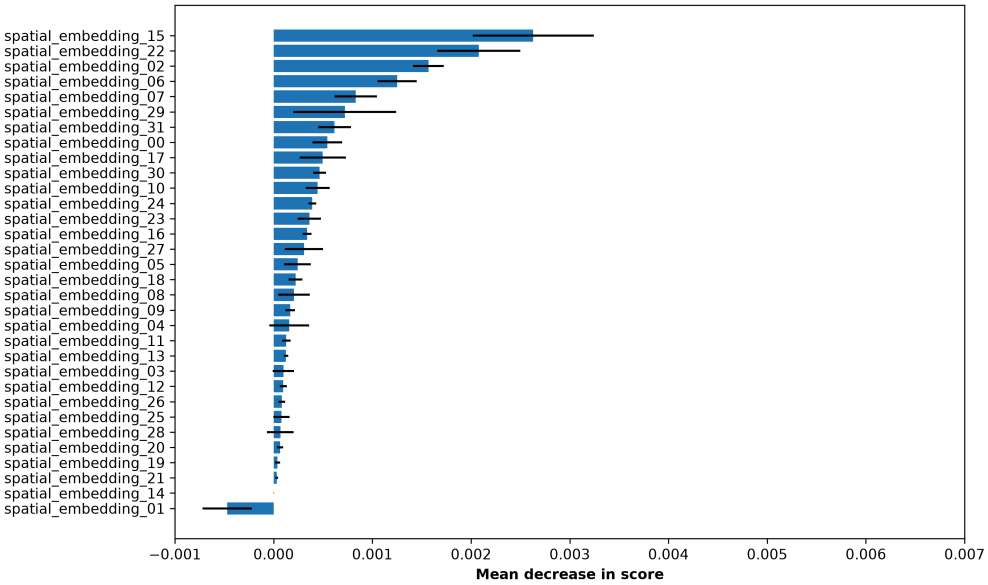


Figure 10: Concede Classifier Permutation Importance: Spatial Embedding Dimensions

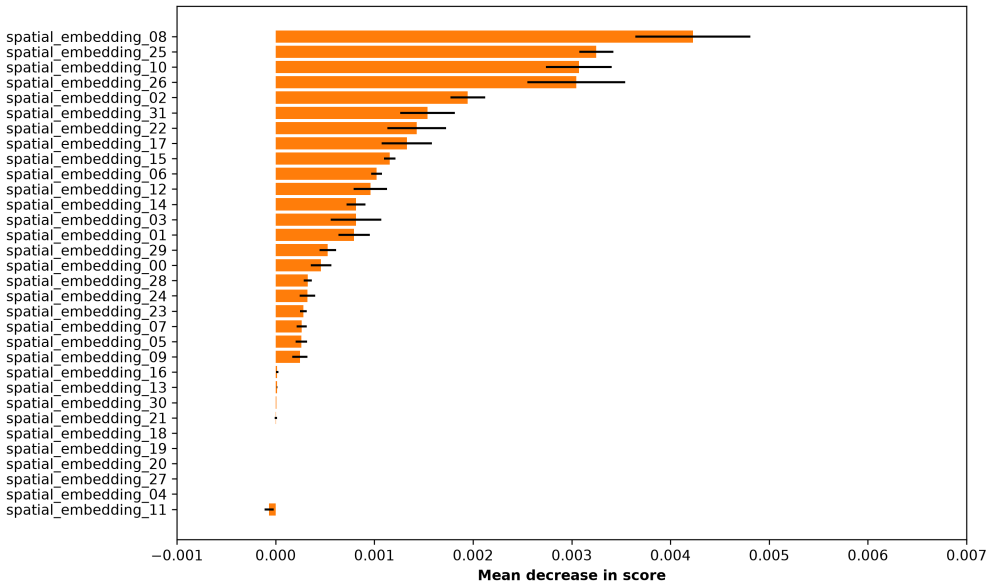


Figure 11: Score Classifier Permutation Importance: Spatial Embedding Dimensions

5.3.2. *Partial Dependence Plots*

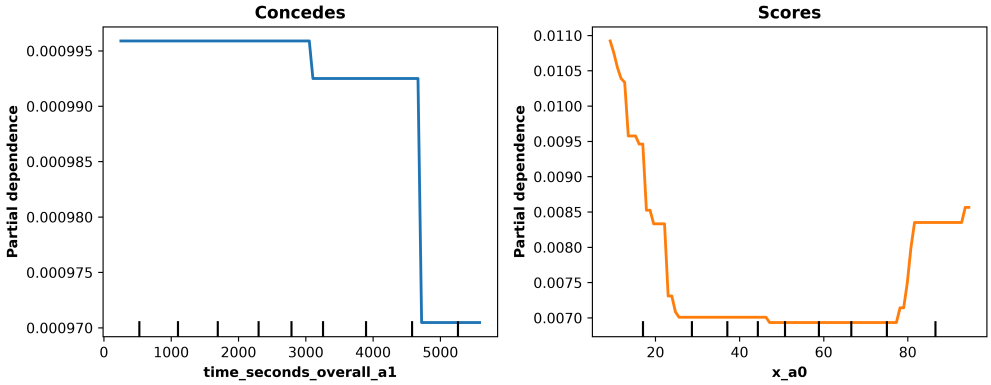


Figure 12: PDP of Most Important Baseline Features for each Model

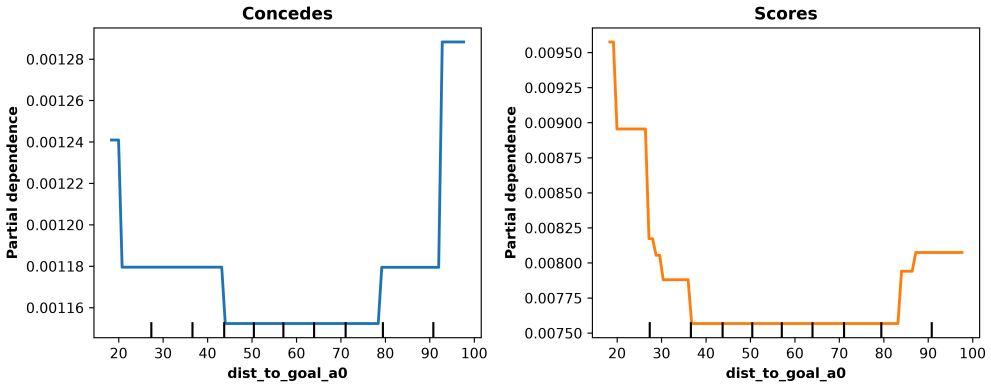


Figure 13: PDP of Distance To Goal

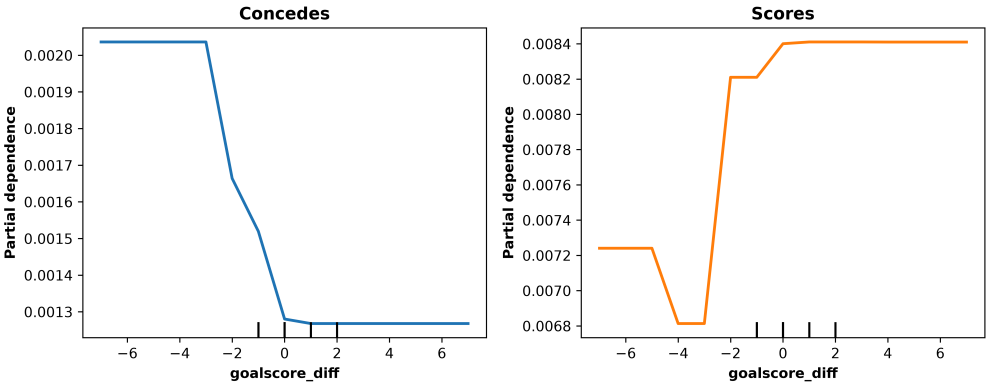


Figure 14: PDP of Goalscore Difference

5.3.3. *K-means Clustering: Spatial Embeddings*

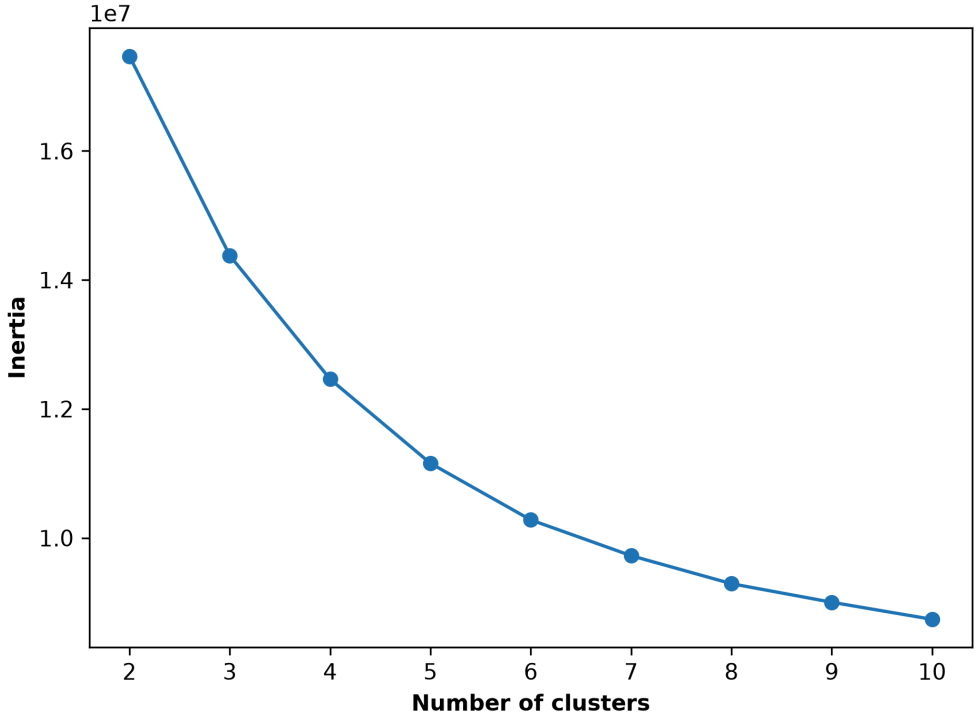


Figure 15: K-means Elbow Method Plot

k	inertia	silhouette	calinski harabasz	davies bouldin
2	1.75e+07	0.26	2.73e+05	1.60
3	1.44e+07	0.30	2.46e+05	1.35
4	1.25e+07	0.29	2.27e+05	1.49
5	1.12e+07	0.29	2.12e+05	1.47
6	1.03e+07	0.29	1.97e+05	1.47
7	9.72e+06	0.29	1.81e+05	1.59
8	9.29e+06	0.28	1.67e+05	1.69
9	9.00e+06	0.27	1.54e+05	1.74
10	8.74e+06	0.27	1.43e+05	1.77

Table 13: K-means Hyperparameter Results

5.3.4. *UMAP*

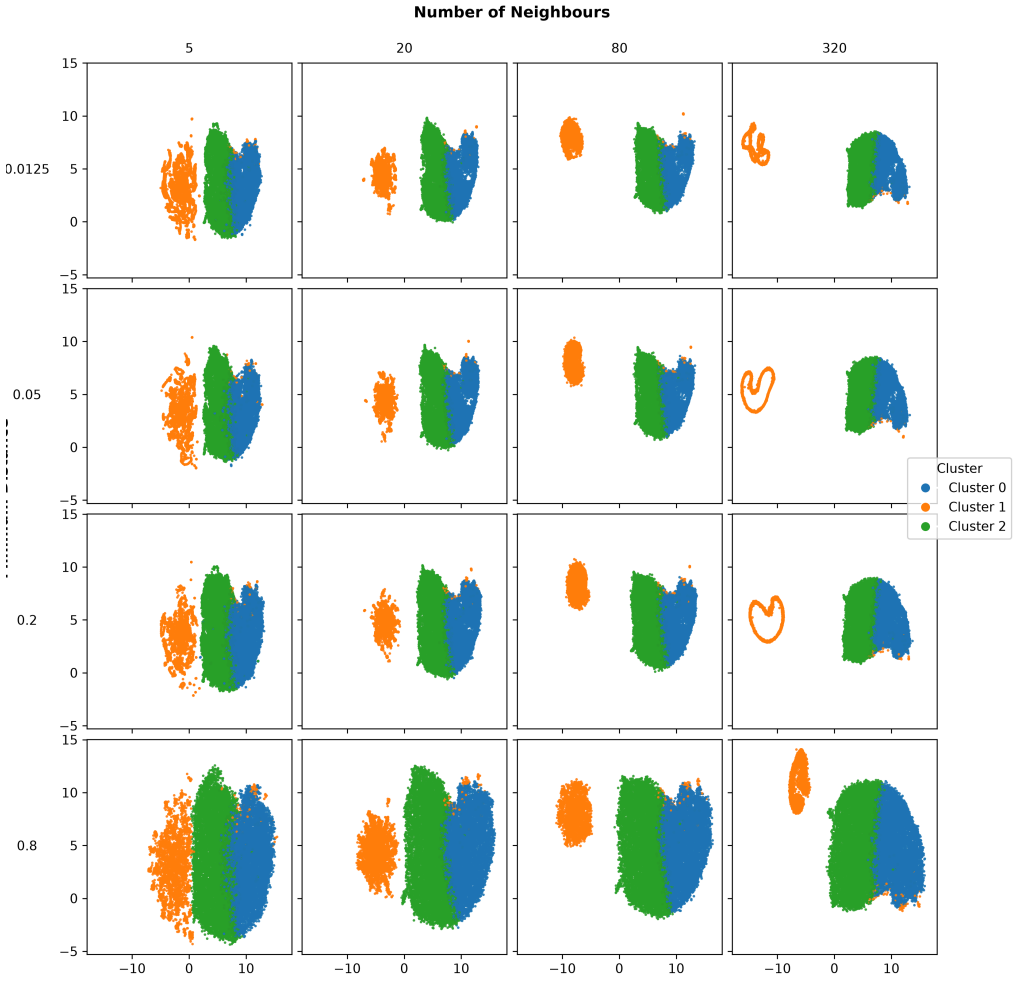


Figure 16: UMAP Coloured by K-means Cluster with Varying UMAP Hyperparameter Values

6. Discussion

7. Conclusion

References

Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019, July 25). *Optuna: A Next-generation Hyperparameter Optimization Framework*. https://doi.org/10.48550/arXiv.1907.10902

Amichay, G., Silva, H., Brito, J., & Marcelino, R. (2025). Characterizing the Spatial Structures of Competing Football Teams. *Scientific Reports*, *15*(1), 35217–35218. https://doi.org/10.1038/s41598-025-97765-y

Anzer, G., & Bauer, P. (2021). A Goal Scoring Probability Model for Shots Based on Synchronized Positional and Event Data in Football (Soccer). *Frontiers in Sports and Active Living*, *3*, 624475–624476. https://doi.org/10.3389/fspor.2021.624475

Ashcroft, A., Das, A., Gryaditskaya, Y., Qu, Z., & Song, Y.-Z. (2023, October 13). *Modelling Complex Vector Drawings with Stroke-Clouds*. The Twelfth International Conference on Learning Representations. https://openreview.net/forum?id=O2jyuo89CK

Bassek, M., Rein, R., Weber, H., & Memmert, D. (2025). An Integrated Dataset of Spatiotemporal and Event Data in Elite Soccer. *Scientific Data*, *12*(1), 195–196. https://doi.org/10.1038/s41597-025-04505-y

Bergstra, J., & Bengio, Y. (2012). Random Search for Hyper-Parameter Optimization. *The Journal of Machine Learning Research*, *13*(null), 281–305. https://dl.acm.org/doi/10.5555/2188385.2188395

Bergstra, J., Bardenet, R., Bengio, Y., & Kégl, B. (2011). Algorithms for Hyper-Parameter Optimization. *Advances in Neural Information Processing Systems*, *24*. https://papers.nips.cc/paper_files/paper/2011/hash/86e8f7ab32cfd12577bc2619bc635690-Abstract.html

Bischofberger, J., Ma, R., Bauer, P., Arnsmeyer, K., & Baca, A. (2026, June 18). *Blame Is Easier than Praise: Measuring off-Ball Defensive Performance in Football*. https://doi.org/10.48550/arXiv.2606.19931

Breiman, L. (2001). Random Forests. *Machine Learning*, *45*(1), 5–32. https://doi.org/10.1023/A:1010933404324

Brier, G. W. (1950). Verification of Forecasts Expressed in Terms of Probability. *Monthly Weather Review*, *78*(1), 1–3. https://doi.org/10.1175/1520-0493(1950)078<0001:VOFEIT>2.0.CO;2

Burriel, B., & Buldú, J. M. (2021). The Quest for the Right Pass: Quantifying Player's Decision Making. *Proceedings of the StatsBomb Innovation in Football Conference*, 1–22. https://statsbomb.wpengine.com/wp-content/uploads/2021/11/Javier-M-Buldu.pdf

Calabuig, J. M., Catalán, C., García-Raffi, L. M., & Sánchez-Pérez, E. A. (2024). A Mathematical Model to Study Defensive Metrics in Football: Individual, Collective and Game Pressures. *Mathematics*, *12*(23), 3854–3855. https://doi.org/10.3390/math12233854

Caliński, T., & Harabasz, J. (1974). A Dendrite Method for Cluster Analysis. *Communications in Statistics*, *3*(1), 1–27. https://doi.org/10.1080/03610927408827101

Capellera, G., Ferraz, L., Rubio, A., Agudo, A., & Moreno-Noguer, F. (2024). Footbots: A Transformer-Based Architecture for Motion Prediction in Soccer. *2024 IEEE International Conference on Image Processing (ICIP)*, 2313–2319. https://doi.org/10.1109/ICIP51287.2024.10647396

Caron, M., Bojanowski, P., Joulin, A., & Douze, M. (2019, March 18). *Deep Clustering for Unsupervised Learning of Visual Features*. https://doi.org/10.48550/arXiv.1807.05520

CatBoost Developers. *CatBoostClassifier | CatBoost*. Retrieved April 22, 2026, from https://catboost.ai/docs/en/concepts/python-reference_catboostclassifier

Cefis, M., & Carpita, M. (2025). A New xG Model for Football Analytics. *Journal of the Operational Research Society*, *76*(1), 1–13. https://doi.org/10.1080/01605682.2024.2323669

Cervone, D., D'Amour, A., Bornn, L., & Goldsberry, K. (2016). A Multiresolution Stochastic Process Model for Predicting Basketball Possession Outcomes. *Journal of the American Statistical Association*, *111*(514), 585–599. https://doi.org/10.1080/01621459.2016.1141685

Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794. https://doi.org/10.1145/2939672.2939785

Coaches' Voice. (2024). *What Is Rest Defence? Football Tactics Explained*. https://learning.coachesvoice.com/cv/rest-defence-explained/

Cox, D. R. (1958). The Regression Analysis of Binary Sequences. *Journal of the Royal Statistical Society. Series B (Methodological)*, *20*(2), 215–242. https://www.jstor.org/stable/2983890

Davis, J., Decroos, T., & Robberechts, P. (2020, May 5). *Introducing Atomic-SPADL: A New Way to Represent Event Stream Data*. https://dtai.cs.kuleuven.be/sports/blog/introducing-atomic-spadl:-a-new-way-to-represent-event-stream-data/

Decroos, T., & Davis, J. (2020). Interpretable Prediction of Goals in Soccer. *Proceedings of the AAAI-20 Workshop on Artifical Intelligence in Team Sports*. AI in Team Sports Organising Committee. https://tomdecroos.github.io/reports/interpret_vaep.pdf

Decroos, T., Bransen, L., Van Haaren, J., & Davis, J. (2019). Actions Speak Louder than Goals: Valuing Player Actions in Soccer. *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 1851–1861. https://doi.org/10.1145/3292500.3330758

Everett, G., Beal, R., Matthews, T., Norman, T., & Ramchurn, G. (2022). Contextual Expected Threat Using Spatial Event Data. *Statsbomb Conference 2022*. https://geverett.co.uk/pdfs/Gregory-Everett-et-al-Contextual-xT-using-Spatial-Event-Data.pdf

Fernandez, J., Bornn, L., & Cervone, D. (2021). A Framework for the Fine-Grained Evaluation of the Instantaneous Expected Value of Soccer Possessions. *Machine Learning*, *110*(6), 1389–1427. https://doi.org/10.1007/s10994-021-05989-6

Fernández, J., Bornn, L., & Fraser, S. (2018). *Wide Open Spaces: A Statistical Technique for Measuring Space Creation in Professional Soccer*. https://www.semanticscholar.org/paper/Wide-Open-Spaces%3A-A-statistical-technique-for-space-Fern%C3%A1ndez-Bornn/a9f6975e93f8c86dadb9249750712305511d5b69

Friedman, J. H. (2001). Greedy Function Approximation: A Gradient Boosting Machine. *The Annals of Statistics*, *29*(5), 1189–1232. https://www.jstor.org/stable/2699986

Goes, F., Meerhoff, L., Bueno, M., Rodrigues, D., Moura, F., Brink, M., Elferink-Gemser, M., Knobbe, A., Cunha, S., Torres, R., & Lemmink, K. (2021). Unlocking the Potential of Big Data to Support Tactical Performance Analysis in Professional Soccer: A Systematic Review. *European Journal of Sport Science*, *21*(4), 481–496. https://doi.org/10.1080/17461391.2020.1747552

Green, S. (2012). *Assessing The Performance of Premier League Goalscorers*. https://www.statsperform.com/resource/assessing-the-performance-of-premier-league-goalscorers/

Jo, G., Kang, M., Lee, K., Lee, M., Bauer, P., & Ko, S.-K. (2026, January 29). *VERSA: Verified Event Data Format for Reliable Soccer Analytics*. https://doi.org/10.48550/arXiv.2601.21981

Kim, H., Choi, H.-J., Kim, C. J., Yoon, J., & Ko, S.-K. (2023). Ball Trajectory Inference from Multi-Agent Sports Contexts Using Set Transformer and Hierarchical Bi-LSTM. *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 4296–4307. https://doi.org/10.1145/3580305.3599779

Kuhn, H. W. (1955). The Hungarian Method for the Assignment Problem. *Naval Research Logistics Quarterly*, *2*(1–2), 83–97. https://doi.org/10.1002/nav.3800020109

Lee, J., Lee, Y., Kim, J., Kosiorek, A. R., Choi, S., & Teh, Y. W. (2019, May 26). *Set Transformer: A Framework for Attention-based Permutation-Invariant Neural Networks*. https://doi.org/10.48550/arXiv.1810.00825

Li, L., Jamieson, K., DeSalvo, G., Rostamizadeh, A., & Talwalkar, A. (2018, June 18). *Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization*. https://doi.org/10.48550/arXiv.1603.06560

Liu, D. C., & Nocedal, J. (1989). On the Limited Memory BFGS Method for Large Scale Optimization. *Mathematical Programming*, *45*(1), 503–528. https://doi.org/10.1007/BF01589116

Macdonald, B. (2012). An Expected Goals Model for Evaluating NHL Teams and Players. *Proceedings of the 2012 MIT Sloan Sports Analytics Conference*, }, % month={March.

McInnes, L., Healy, J., & Melville, J. (2020, September 18). *UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction*. https://doi.org/10.48550/arXiv.1802.03426

Mead, J., O'Hare, A., & McMenemy, P. (2023). Expected Goals in Football: Improving Model Performance and Demonstrating Value. *PLOS ONE*, *18*(4), e282295. https://doi.org/10.1371/journal.pone.0282295

Naeini, M. P., Cooper, G., & Hauskrecht, M. (2015). Obtaining Well Calibrated Probabilities Using Bayesian Binning. *Proceedings of the AAAI Conference on Artificial Intelligence*, *29*(1). https://doi.org/10.1609/aaai.v29i1.9602

Olthof, S., & Davis, J. (2025). Perspectives on Data Analytics for Gaining a Competitive Advantage in Football: Computational Approaches to Tactics. *Science and Medicine in Football*, *0*(0), 1–13. https://doi.org/10.1080/24733938.2025.2533784

Parzen, E. (1962). On Estimation of a Probability Density Function and Mode. *The Annals of Mathematical Statistics*, *33*(3), 1065–1076. https://www.jstor.org/stable/2237880

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-Learn: Machine Learning in Python. *J. Mach. Learn. Res.*, *12*(null), 2825–2830. https://dl.acm.org/doi/10.5555/1953048.2078195

Peters, A. J., Parmar, N., Davies, M., Reeves, M., Sormaz, M., & James, N. (2024). Expected Pass Turnovers (xPT) - a Model to Analyse Turnovers from Passing Events in Football. *Journal of Sports Sciences*, *42*(11), 1002–1010. https://doi.org/10.1080/02640414.2024.2379697

Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2019, January 20). *CatBoost: Unbiased Boosting with Categorical Features*. https://doi.org/10.48550/arXiv.1706.09516

Robberechts, P., & Davis, J. (2020). *How Data Availability Affects the Ability to Learn Good xG Models* (pp. 17–27). https://doi.org/10.1007/978-3-030-64912-8_2

Rosenblatt, M. (1956). Remarks on Some Nonparametric Estimates of a Density Function. *The Annals of Mathematical Statistics*, *27*(3), 832–837. https://www.jstor.org/stable/2237390

Rousseeuw, P. (1987). Silhouettes: A Graphical Aid to the Interpretation and Validation of Cluster Analysis. *Journal of Computational and Applied Mathematics*, *20*, 53–65. https://doi.org/10.1016/0377-0427%2887%2990125-7

Schaerlaeckens, L. (2025). The Belgian Lab Shaping Modern Soccer's Data Revolution. *The Guardian*. https://www.theguardian.com/football/2025/jun/03/soccer-analytics-jesse-davis-leuven

Schubert, E. (2023). Stop Using the Elbow Criterion for K-Means and How to Choose the Number of Clusters Instead. *ACM SIGKDD Explorations Newsletter*, *25*(1), 36–42. https://doi.org/10.1145/3606274.3606278

scikit-learn Developers. *RandomForestClassifier*. Retrieved April 22, 2026, from https://scikit-learn/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html

Singh, K. (2018). *Introducing Expected Threat (xT)*. https://karun.in/blog/expected-threat.html

Socceraction Developers. *Socceraction 1.5.3 Documentation*. Retrieved April 22, 2026, from https://socceraction.readthedocs.io/en/latest/documentation/intro.html

Spearman, W., Basye, A., Dick, G., Hotovy, R., & Pop, P. (2017, March 3). *Physics-Based Modeling of Pass Probabilities in Soccer*. https://www.researchgate.net/publication/315166647_Physics-Based_Modeling_of_Pass_Probabilities_in_Soccer

StatsBomb. (2026a). *StatsBomb Open Data* [Computer software]. StatsBomb. https://github.com/statsbomb/open-data

StatsBomb. (2026b, July 6). *Statsbomb/Statsbombpy*. https://github.com/statsbomb/statsbombpy

Sá-Freire, L. M. de. (2025). Event Data Based Action Valuation: Possibilities and Limitations. In D. Memmert (Ed.), *Artificial Intelligence and Machine Learning in Sports Science* (pp. 285–292). Springer. https://doi.org/10.1007/978-3-662-70155-3_20

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). *Attention Is All You Need*. https://doi.org/10.48550/arXiv.1706.03762

Wakelam, E., Steuber, V., & Wakelam, J. (2022). The Collection, Analysis and Exploitation of Footballer Attributes: A Systematic Review. *Journal of Sports Analytics*, *8*(1), 31–67. https://doi.org/10.3233/JSA-200554

Wales, D. J., & Doye, J. P. K. (1997). Global Optimization by Basin-Hopping and the Lowest Energy Structures of Lennard-Jones Clusters Containing up to 110 Atoms. *The Journal of Physical Chemistry a*, *101*(28), 5111–5116. https://doi.org/10.1021/jp970984n

Xie, E., Wang, W., Yu, Z., Anandkumar, A., Alvarez, J. M., & Luo, P. (2021, May 31). *SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers*. https://arxiv.org/abs/2105.15203v3